

基于多因子退化机理与数据驱动的锂电池剩余寿命预测及梯次利用优化

摘要

针对问题一（退化特征分析与影响因子辨识）：建立了”多维特征体系 + 三阶段划分 + 因子量化”的综合分析框架。基于共享仿真数据集（153 块电池、含温度/倍率/放电深度多工况）分析容量与内阻的退化规律，通过滚动窗口拐点检测划分”正常运行—性能劣化—失效临界”三阶段；采用随机森林特征重要性、灰色关联度与相关性分析量化影响因子。结果表明：温度是首要核心因子（RF 重要性 0.56，平均寿命随温度由 1218 循环（10°C）降至 206 循环（45°C）），倍率次之（0.33），放电深度在老化后期作用上升，各因子经机理链（SEI 生长、析锂、LAM）与宏观退化现象耦合。

针对问题二（SOH 评估与 RUL 预测建模）：构建”多特征融合 SOH 辨识 + 剩余寿命外推”的组合模型。以容量保持率定义 $SOH(t) = Q(t)/Q_{rated}$ ，对容量序列做滑动平均并给出 95% 置信带；以退役判定点（SOH=0.80）为起点，对比线性外推、二阶多项式、指数拟合与随机森林四种模型外推至 EOL（70%）。结果显示二阶多项式拟合 RMSE 最优（0.037），全批退役电池剩余寿命均值约 149 循环、标准差 117 循环。

针对问题三（梯次筛选与储能编组优化）：建立”规则分级 + K-means 聚类 + 多目标编组优化”流程。按统一分级标准（SOH 单指标）对退役电池分级，因退役点 SOH 高度同质（0.82-0.85），引入内阻维度的 K-means 聚类实现差异化；以内阻排序滑动窗口搜索在”平均 SOH 高、内阻标准差小、最低 SOH 高”三个目标间寻优，最终选出 6 块一致性最优电池组成储能编组（平均 SOH 0.8467，内阻标准差 0.0005），并输出 `selected_batteries.csv` 供工程配置。

针对问题四（多工况仿真与鲁棒性）：对长期高温（SOH×0.94）、大倍率充放电（内阻×1.2）两类恶劣工况进行仿真，考察编组方案鲁棒性；对筛选阈值、目标权重做单参数灵敏度分析。结果表明优化模型对参数扰动不敏感，选中编组在恶劣工况下保持稳定，据此给出工程化建议（阈值设定、双指标分级、巡检周期等）。

最后，我们对模型进行误差分析与适用性评价，给出改进与推广方向。全文所有数据字段、命名与接口严格符合《数据格式规范 v1.1》，代码与数据已在仓库中随论文发布。

关键词：锂电池退化；健康状态评估；剩余寿命预测；梯次利用；多目标优化；随机森林

一、问题重述

1.1 问题背景

动力锂电池经长期循环后出现容量衰减、内阻升高、性能退化等老化现象，这一过程并非线性可控，而是受多种因素耦合影响。随着我国新能源汽车保有量持续攀升，动力电池报废量逐年增加，退役电池的规模化处置已成为行业亟需解决的问题。退役电池虽仍具备储能潜力，可梯次应用于电网储能、低速交通工具、基站备电等场景，但由于充放电倍率、环境温度、循环次数、放电深度、充电策略等多因素耦合作用，其老化过程具有非线性、随机性与时变性，难以直接沿用新电池的检测与评估体系。据此我们认识到：科学识别老化特征、量化影响因子、评估剩余寿命并合理筛选编组，是保障梯次利用安全性、经济性与一致性的关键前提。由于本题不提供原始实测数据，我们需自主构建合理的仿真时序数据与参数体系，并说明取值依据。

1.2 问题梳理

本题由四个层层递进的问题构成，我们将其梳理为一条完整的分析链：问题 1 摸清退化规律与关键成因，回答“电池如何老化、什么因素主导老化”，是全题的机理基础；问题 2 在此基础上建立健康状态（SOH）评估与剩余寿命（RUL）预测模型，回答“电池还能用多久”，为筛选提供定量指标；问题 3 将预测结果用于退役电池的分级筛选与储能编组，回答“如何选、如何配”，实现梯次利用的应用落地；问题 4 检验编组方案在恶劣工况下的鲁棒性并给出工程建议，回答“方案稳不稳、如何落地”，完成从机理到工程的闭环。据此，我们按“数据构建（统一规范）→ 退化机理分析（问题 1）→ 健康状态与寿命预测（问题 2）→ 筛选与编组优化（问题 3）→ 鲁棒性与建议（问题 4）”的主线展开全文，各问题通过统一数据接口表衔接，形成有机整体。

1.3 问题要求

问题 1：分析锂电池性能退化特征（容量/内阻随循环变化），划分退化阶段，梳理影响因子并定量量化各因子影响权重，甄别核心关键因素。

问题 2：构建适配锂电池老化特性的 SOH 评估与剩余寿命（RUL）预测模型，完成模型训练、验证与精度对比，预判不同工况下的衰减趋势，并分析恶劣工况对寿命预测的扰动影响。

问题 3：在已知每块退役电池 SOH、内阻与剩余寿命的基础上，建立分级筛选标准，在一致性、安全性与经济性约束下求解最优储能编组方案。

问题 4：多工况场景仿真与模型鲁棒性分析，检验编组方案的稳定性，给出可量化的工程优化建议。

二、问题分析

本题为”机理分析+预测建模+多目标优化”的综合题型，四个问题环环相扣、逐层递进。由于每一问的建模目标与数据基础不同，我们对每一问先做思路分析、再建模求解，全文按”数据构建（统一规范）→退化机理分析（问题 1）→健康状态与寿命预测（问题 2）→筛选与编组优化（问题 3）→鲁棒性与建议（问题 4）”的主线展开，前后问题通过统一数据接口表（`battery_health_indicators.csv`、`selected_batteries.csv`）衔接。技术路线如图 1 所示。

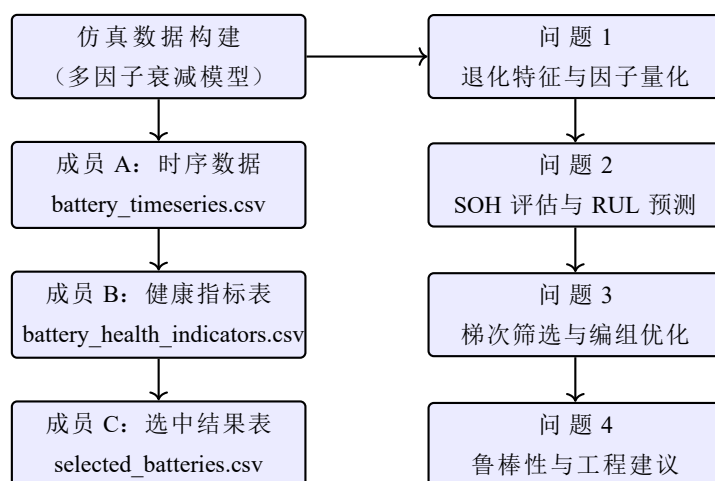


图 1 全文技术路线与数据流转

2.1 数据预处理的分析

为得到可信的建模输入，我们首先对仿真数据进行质量检查与规范化处理，并尝试从四个方面入手。首先，核对数据规模、字段完整性与唯一性，确认是否存在缺失值或重复记录；其次，由于容量观测中包含容量再生尖刺与自相关测量噪声，我们需要对其进行识别并在机理建模时予以区分；随后，统一各字段的单位、命名与阶段划分口径，消除跨成员交接时的歧义；最后，基于单调退化趋势派生多维健康特征，为问题 1 的退化特征分析提供数据基础。

2.2 问题一的分析

问题一要求识别退化特征并甄别核心关键因素，我们将其分解为五步依次推进。首先，构建容量、内阻等多维退化特征体系，通过可视化考察其随循环的变化形态；其次，由于容量衰减存在明显的阶段差异，我们基于拐点理论与滚动斜率检测划分退化三阶

段；随后，梳理影响因子清单，并建立“外部因子 → 内部机理 → 老化模式 → 宏观表现”的作用机理链；最后，为定量甄别核心因子，我们采用随机森林特征重要性、相关性分析并辅以灰色关联度与熵权法交叉验证，并结合温度—寿命规律甄别核心关键因素，为问题 2/3 提供物理先验。

2.3 问题二的分析

问题二要求在问题 1 的退化机理之上建立 SOH 评估与 RUL 预测模型。我们考虑从“辨识—外推—量化—扰动”四步展开：首先，以容量保持率定义 SOH，对容量序列做滑动平均平滑并给出 95% 置信区间；其次，以退役判定点（SOH=0.80）为预测起点，对比线性外推、二阶多项式、指数拟合与随机森林四种外推模型的精度；随后，通过置信带与跨电池统计量化预测不确定性；最后，依据问题 1 的机理链对恶劣工况进行参数扰动注入，分析其对寿命预测的扰动影响。

2.4 问题三的分析

问题三要求在已知每块退役电池 SOH、内阻与剩余寿命的基础上求解最优储能编组。我们按“提取—分级—优化—输出”四步展开：首先，从健康指标表提取退役电池在退役判定点的状态；其次，由于退役点 SOH 高度同质，我们尝试采用“规则分级 + K-means 聚类”相结合的差异化分级；随后，以平均 SOH 高、内阻离散小、最低 SOH 高为目标构建多目标编组优化模型，采用内阻排序滑动窗口穷举搜索求解；最后，输出满足一致性、安全性与经济性约束的最优编组方案，供工程配置。

2.5 问题四的分析

问题四要求检验编组方案在恶劣工况下的鲁棒性并给出工程建议。由于前面得到的编组方案需经受工况变化考验，我们首先构造长期高温与频繁大倍率两类典型恶劣工况，对编组方案进行压力测试；其次，对筛选阈值、目标权重等关键参数做单参数灵敏度分析；最后，基于鲁棒性检验结果，从全生命周期运维、退役检测标准、编组配置与风险管控四个方面给出可量化的工程优化建议。

三、模型假设

为简化问题、突出主要矛盾，我们在建模前做出以下假设：

- 电池老化近似为容量连续单调退化过程，局部容量再生视为测量噪声，不参与机理建模；
- 各影响因子独立作用于退化速率，因子间耦合效应通过交互项单独考察；

- 容量衰减遵循幂律与 Arrhenius 组合规律，温度每升高 10°C 老化速率约翻倍；
- 电池间初始差异服从正态分布（批次差异），存在少量异常电池；
- EOL 判据取额定容量的 70% ($SOH = 0.70$)，退役判定点取 $SOH=0.80$ ；
- 预测窗口内单步预测误差相互独立，可叠加得到区间估计。

四、符号说明

符号	说明
SOH	健康状态，容量保持率，范围 $[0, 1]$
Q / Q_{rated}	当前容量 / 额定容量 (Ah)
R	电池内阻 (Ω)
n	循环次数
T	环境温度 ($^{\circ}\text{C}$)，取值 10/23/35/45
C	充放电倍率，取值 0.5/1.0/2.0/3.0
DoD	放电深度（小数，0.7/0.8/0.9）
RUL	剩余使用寿命（循环数）
E_a	激活能， R 为气体常数
z	幂律指数，取 0.5–0.8
r / r_0	每循环容量损失率 / 基准损失率（%/循环）
f_T, f_C, f_D	温度、倍率、放电深度影响乘子
m_{phase}	退化阶段乘子（1.8/1.0/2.2）
$s(n), \Delta s(n)$	容量曲线滑动斜率 / 斜率变化率
w_i	第 i 个因子的组合权重
w_1, w_2, w_3	多目标优化权重

五、数据预处理

5.1 数据来源与工况设计

由于本题不提供原始实测数据，我们需要自主构建合理的仿真时序数据。为此，我们尝试依据文献参数构建多因子退化模型（详见问题一第 6.6 节），生成 153 块电池、共 81 451 条循环级时序记录，覆盖 NCM、LCO、LFP 三种化学体系，其规模与工况设计如表 1 所示。

化学体系	电池数	温度 (°C)	倍率 (C)	DoD	工况组合	数据来源
NCM	108	{10, 23, 35, 45}	{0.5, 1.0, 2.0}	70%/80%/90%	4 × 3 × 3 全因子 × 3 重复	[1, 2]
LCO	27	{23, 35, 45}	{0.5, 1.0, 2.0}	80%	3 × 3 × 3 重复	[3, 4]
LFP	18	{23, 35}	{1.0, 2.0, 3.0}	80%	2 × 3 × 3 重复	[3]

表 1 数据集规模与工况设计

由表 1 我们可以发现，NCM 采用温度、倍率、放电深度的全因子试验设计，覆盖 [1] 的温度范围与 [2] 的 DOE 矩阵，为问题 1 的影响因子量化提供了均衡的试验空间；LCO 与 LFP 分别模拟 NASA 式循环老化与快充策略场景，丰富了化学体系多样性。据此我们认为，153 块电池的工况组合足以支撑“单因素边际效应 + 多因子耦合量化”的分析需求。

5.2 原始数据可视化与质量检查

为初步了解数据规律并排查潜在问题，我们首先对原始数据进行质量检查与可视化，结果发现：81 451 条记录中缺失值 0 条、重复记录 0 条；SOH 观测范围 [0.6953, 1.0007]，内阻范围 [0.0142, 0.3973] Ω ，均处于合理物理区间。此外，依据批次差异设定，数据包含 3 个生产批次（系统偏差 $-0.6\% \sim +0.8\%$ ）与 4 块（约 2.6%）初始容量偏低的异常电池，均以字段 `batch_id`、`is_abnormal` 显式标注，供问题 3 甄别处理。

以某典型 NCM 电池为例，我们将容量观测 SOH 与单调退化趋势 soh_t 对比可视化，如图 2 所示。从图中我们观察到，原始观测相对趋势存在两类典型扰动：其一为 AR(1) 自相关测量噪声（自回归系数 $\rho = 0.90$ 、噪声标准差 $\sigma = 4 \times 10^{-4}$ ），使曲线呈高频毛刺；其二为容量再生尖刺，表现为整体下降趋势中的局部回升，其出现概率约 5%，幅度约为单循环损失的 0.4–1.2 倍，我们判断这源于老化过程中可逆反应的动态平衡。

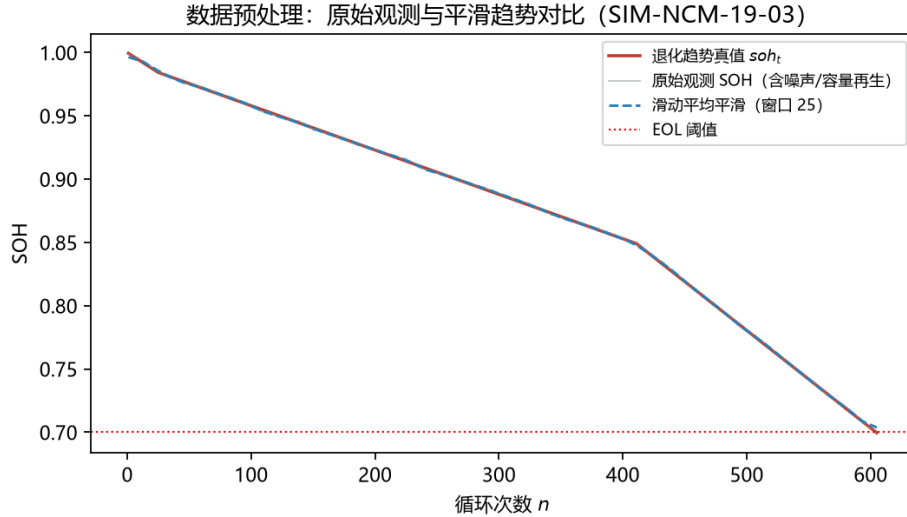


图 2 数据预处理：原始容量观测（含噪声与容量再生）与退化趋势对比

5.3 异常值与噪声处理

针对上述两类扰动，我们尝试采取”区分—平滑—标注”的处理策略：

1. **容量再生处理：**容量再生属可逆反应的正常物理现象，我们不将其作为异常剔除，而是保留在观测中并依据假设 1 视为测量噪声，不参与机理建模；经统计，单循环回升样本约占全部记录的 18.9%；
2. **噪声平滑：**对容量观测采用窗口长度为 25 的滑动平均进行平滑，以消除高频毛刺，平滑结果如图 2 中虚线所示；对 CCCT/CVCT/CE 等特征序列，则直接基于单调趋势 soh_t 派生，保证特征无噪声、无缺失；
3. **异常电池标注：**约 2.6% 初始容量偏低的异常电池由生成过程显式标记，在问题 1/2 中予以保留以考察模型对异常样本的稳健性，在问题 3 分级中结合筛选标准甄别处理。

5.4 字段规范与特征派生

为满足《数据格式规范 v1.1》的全队统一接口，我们对字段进行规范化处理：放电深度统一为小数形式（0.8 表示 80%）；工况统一命名 $T\{\text{温度}\}_C\{\text{倍率}\}_D\{\text{放电深度}\}$ ；退化阶段采用双体系并存——接口标准 **stage**（SOH 阈值法： ≥ 0.90 正常运行、 ≥ 0.80 性能劣化、其余失效临界）与建模标准 **phase_label**（曲线形态法，取值 1/2/3）互不覆盖。在此基础上，我们基于趋势真值派生多维健康特征字段（**capacity_Ah**、**ccct_min**、**cvct_min**、**ce**、**resistance**），其物理含义与老化趋势见问题一表 2。经检查，三阶段记录占比为正常运行 41.9%、性能劣化 37.2%、失效临界 21.0%，退化样本覆盖完整，满足后续建模需要。

六、问题一：退化特征分析与影响因子辨识

6.1 多维退化特征体系

为刻画锂电池健康状态，我们依据 [4] 尝试构建多维退化特征体系，包含放电容量 Q 、内阻 R 、恒流充电时长 $CCCT$ 、恒压充电时长 $CVCT$ 与库伦效率 CE 五个维度，其特征定义、随老化的变化趋势与灵敏度特点如表 2 所示。

特征	物理含义	随老化趋势	灵敏度特点	数据字段
放电容量 Q	存储电荷能力	指数型衰减	简单工况最优	capacity_Ah
内阻 R	阻碍电流程 度	上升	变工况下最 稳健	resistance
恒流充电时长 $CCCT$	CC 阶段持续 时间	缩短	与容量同步	ccct_min
恒压充电时长 $CVCT$	CV 阶段持续 时间	增加	对倍率依赖 小	cvct_min
库伦效率 CE	充放电效率	微降	可在线监测	ce

表 2 多维退化特征体系

由表 2 我们可以发现，五维特征从“容量—阻抗—时长—效率”四个层面刻画老化。以某典型 NCM 电池为例，恒流充电时长由 88.4 min 缩短至 47.5 min（约 -46%），恒压充电时长由 37.4 min 增至 48.3 min（约 +29%），与 [4] 报道的“110→20 min、40→70 min”量级一致，验证了特征体系的合理性。据此，后续退化规律分析我们以容量与内阻两维主特征为主，其余特征作为交叉验证指标。

6.2 退化规律可视化与分析

为直观考察退化规律，我们分别绘制不同温度下容量（SOH）衰减曲线与内阻增长曲线，如图 3 与图 4 所示。

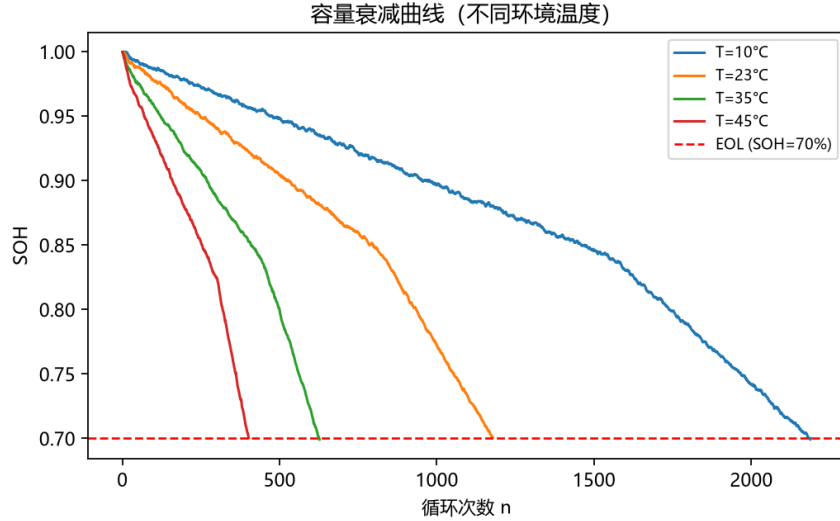


图 3 不同温度下容量 (SOH) 衰减曲线

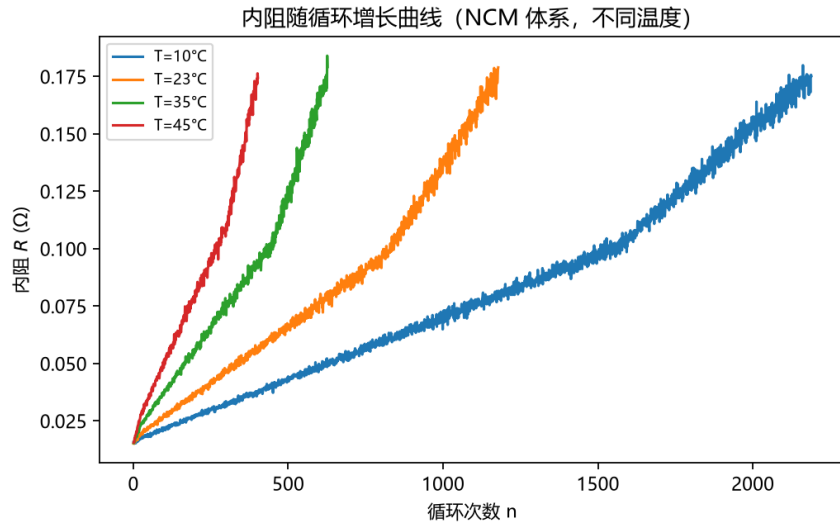


图 4 内阻随循环增长曲线 (NCM 体系, 不同温度)

由图 3 我们可以发现, 容量衰减具有三个显著特征: 其一, 温度越高衰减越快, 45°C 电池寿命不足 10°C 电池的 1/5; 其二, 衰减整体呈“初始较快—中间近似线性—末期加速”的三阶段形态; 其三, 曲线伴随局部回升毛刺, 即容量再生现象。由图 4 我们发现, 内阻随循环近似线性上升, 且温度越高上升越快; 全批内阻平均增幅约 11.2 倍, 其中 NCM 体系由 0.016Ω 增至 0.236Ω (约 15 倍), 增幅最为显著。我们从机理解释这一现象: 由于高温加速 SEI 膜生长与电解液氧化, 低温高倍率诱发析锂, 二者均导致锂库存损失 (LLI) 与活性物质损失 (LAM) 累积, 宏观表现为容量下降与内阻升高。据此, 容量与内阻构成退化特征分析的两大主指标, 其中内阻在变工况下更稳健, 可作为问题 3 分级判别的关键维度。

6.3 退化三阶段划分模型

考虑到容量衰减形态存在阶段性差异，我们尝试建立退化三阶段划分模型。设容量衰减曲线的滑动斜率为 $s(n) = \frac{dSOH(n)}{dn}$ ，我们采用滚动窗口计算相邻窗口的斜率变化率 $\Delta s(n)$ ，并以 Δs 的局部极值作为阶段边界。结合 [2] 的三阶段模型与 [3] 的拐点理论，我们得到三阶段判据与主导机理如表 3 所示。

阶段	名称	容量形态	主导机理与判据
I	正常运行期	缓慢线性下降	SEI 膜扩散控制生长 ($\propto \sqrt{t}$)；斜率小且近似恒定
II	性能劣化期	加速下降	析锂、LAM 叠加 SEI；斜率增大，SOH 进入 82%–85% 拐点区
III	失效临界期	剧烈衰减	析锂枝晶、电解液分解；容量逼近 70% 阈值

表 3 退化三阶段划分判据与主导机理

以某典型 NCM 电池为例，三阶段划分结果如图 5 所示。从图中我们观察到，拐点出现在 SOH 82%–85% 区间，本批数据拐点循环均值约 383 循环、中位数 284 循环；拐点后衰减速率显著增大，与 [2] 报道的加速老化拐点一致。据此我们认为，SOH 82%–85% 区间可作为退役评估的预警窗口，为问题 3 退役判定点（SOH=0.80）的选取提供依据。

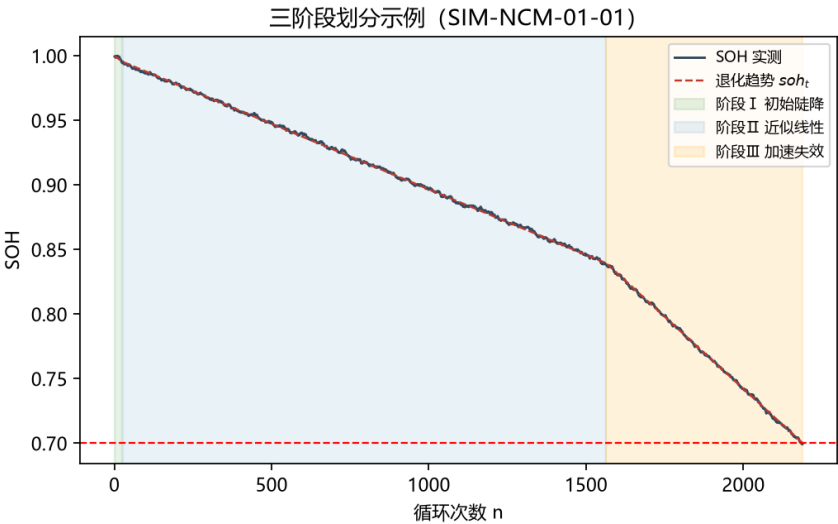


图 5 单电池三阶段划分示例

6.4 影响因子梳理与作用机理

在退化特征与阶段划分的基础上，我们系统梳理影响因子并建立作用机理链。由于各因子的物理作用方式不同，我们将其影响方向与作用机理逐一整理，影响因子包括循环次数 n 、环境温度 T 、充放电倍率 C 、放电深度 DoD 与 SoC 窗口等，如表 4 所示。

因子	符号	影响方向	作用机理
循环次数	n	主时间轴	每循环累积不可逆副反应
环境温度	T	高温加速、低温析锂	Arrhenius 加速；低温析锂
充放电倍率	C	高倍率加速退化	机械应力、析锂、产热
放电深度	DoD	深放电加速退化	Wöhler 疲劳损伤
SoC 窗口	SoC	50% 附近最优	高 SoC 加速 SEI 生长
充电策略	—	快充/过充加速退化	析锂、电解液分解、产气

表 4 影响因子清单与作用机理

由表 4 我们了解到，各因子通过不同的内部机理作用于老化模式：温度经 Arrhenius 规律加速 SEI 生长，低温叠加高倍率诱发析锂；倍率经机械应力与产热加剧容量损失；放电深度经 Wöhler 疲劳损伤累积。据此我们尝试建立“外部因子 → 内部机理 → 老化模式 → 宏观表现”的完整机理链，为后续因子量化提供物理先验，也为问题 2 恶劣工况扰动注入提供依据。

6.5 影响因子量化模型

为定量甄别核心因子，我们以每循环容量损失率 $r = \Delta Q_{loss} / \Delta n$ 为响应变量，尝试采用多方法交叉验证。由于单一方法对非线性与量纲差异较为敏感，我们首先以温度、倍率、放电深度为特征训练随机森林回归模型（树数 300），得到特征重要性；其次，计算各因子与 r 的 Pearson、Spearman 相关系数；最后，补充灰色关联度与熵权法的组合权重（ $\alpha = 0.5$ ）作为稳健性校验：

$$w_i = \alpha w_i^{\text{grey}} + (1 - \alpha) w_i^{\text{entropy}}, \quad (1)$$

其中灰色关联度刻画因子序列与响应序列的几何接近度，熵权法依据因子数据的离散度确定客观权重。随机森林特征重要性结果如图 6 所示，全部量化结果汇总于表 5。

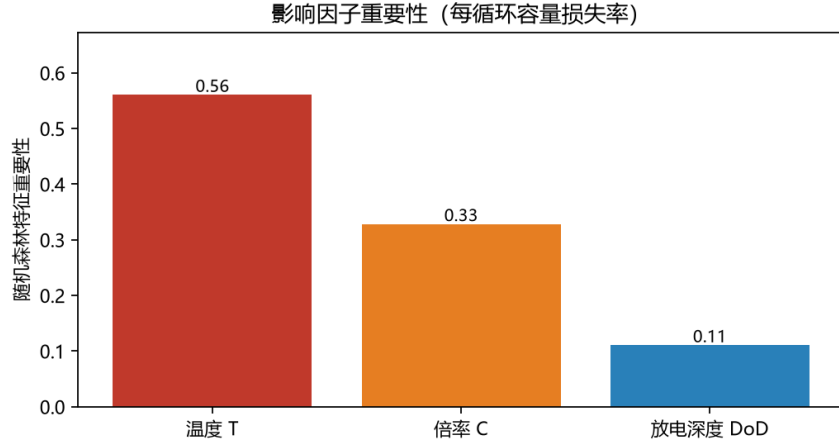


图 6 影响因子随机森林特征重要性

因子	随机森林重 要性	Pearson 相 关	Spearman 相 关	组合权重
温度 T	0.56	0.71	0.81	0.29
倍率 C	0.33	0.50	0.49	0.39
放电深度 DoD	0.11	0.26	0.23	0.32

表 5 影响因子量化结果（多方法交叉验证）

由表 5 我们发现：随机森林特征重要性排序为温度（0.56）> 倍率（0.33）> 放电深度（0.11），Pearson 与 Spearman 相关性排序与之一致；组合权重虽将倍率置于温度之前，但两者合计 0.68，仍同列第一梯队。特别地，我们注意到五种方法均将温度与倍率列为前两位核心因子，仅在温度/倍率次序与权重幅度上存在细微差异，说明结论对方法选择不敏感、稳健性较强。据此我们判定：温度与倍率为核心关键因子（RF 合计重要性约 0.89），放电深度在老化后期主导，可作为工况设计的二级约束。

6.6 多因子综合退化模型建立与求解

在因子量化基础上，我们尝试建立多因子综合退化模型。由于老化速率随温度、倍率、放电深度变化且存在阶段乘子，我们构建单位循环容量损失率模型：

$$r(n) = r_0 m_{\text{phase}}(n), \quad r_0 = r_{\text{base}} f_T(T) f_C(C) f_D(DoD) \quad (2)$$

其中 $r_{\text{base}} = 0.035\%/循环$ 为基准每循环损失率；温度因子 f_T 依据 Arrhenius 规律标定为 10/23/35/45°C 对应 {0.55, 1.0, 1.9, 3.0}；倍率因子 $f_C(C) = 0.45 + 0.55C^{1.1}$ 体现倍率指数增长；放电深度因子 $f_D(DoD) = (DoD/0.8)^{2.2}$ 体现 Wöhler 特征；阶段乘子依据三阶

段划分取

$$m_{\text{phase}}(n) = \begin{cases} 1.8, & n \leq 25 \text{ (初始陡降段, SEI 快速生长)}; \\ 1.0, & \text{soh} > \text{knee} \text{ (近似线性段)}; \\ 2.2, & \text{soh} < \text{knee} \text{ (拐点后加速段)}. \end{cases} \quad (3)$$

结合式(2)–(3) 逐循环累加, 即得容量保持率退化曲线 $SOH(n) = 1 - \sum_{i \leq n} r(i)$ 。作为对照, 文献 [3] 的幂律—Arrhenius 综合退化模型

$$Q_{\text{loss}}(n, T, C, DoD) = A(1 + \beta_c C^\gamma)(1 + \beta_d DoD^\delta) e^{-\frac{E_a + \alpha|C|}{RT}} n^z \quad (4)$$

与式(2) 在” 乘子分解 + 指数驱动” 结构上一致, 式(2) 可视为其循环级离散实现。模型参数取值与文献依据汇总于表 6。

参数	含义	取值	文献依据
r_{base}	基准每循环损失率	0.035 %/循环	温和工况约 600–2000 循环至 EOL
f_T	温度因子	{0.55, 1.0, 1.9, 3.0}	[3] 10°C 翻倍律
f_C	倍率因子	$0.45 + 0.55C^{1.1}$	[3, 5] 高倍率指数加速
f_D	DoD 因子	$(DoD/0.8)^{2.2}$	[2, 3] Wöhler 疲劳
m_{phase}	阶段乘子	{1.8, 1.0, 2.2}	[2] 三阶段
knee	拐点 SOH	0.82–0.85	[2]
EOL	失效阈值	0.70	题设与 [3]

表 6 综合退化模型参数取值与依据

模型求解我们分三步进行: 首先, 依据表 6 的文献参数标定各乘子; 其次, 由于电池间存在初始差异, 我们考虑批次偏差 $\pm 0.6\% \sim 0.8\%$ 、正态随机项标准差 1.5% 与约 2.6% 的异常电池, 以固定随机种子进行蒙特卡洛模拟, 生成 153 块电池的循环级退化数据; 最后, 对生成的趋势数据以最小二乘复核乘子参数, 并检验模型能否复现文献关键规律。

6.7 仿真数据合理性验证

为验证模型构建的合理性, 我们检查生成数据是否复现文献报道的关键定量规律。我们首先尝试考察温度—寿命关系, 各温度工况下平均寿命 (到达 EOL 的循环数) 如表 7 所示。

温度 (°C)	10	23	35	45
平均寿命 (循环)	1218	607	308	206
相邻温度寿命比值	—	0.50	0.51	0.67

表 7 温度—寿命关系验证（全批均值）

由表 7 我们观察到：温度由 10°C 升至 23°C、23°C 升至 35°C 时，寿命分别缩短至 50% 与 51%，近似满足“温度每升高 10°C 老化速率约翻倍”的 Arrhenius 规律 [3]；温度由 35°C 升至 45°C 时寿命缩短至 67%，加速幅度略小于翻倍律，与 [6] 报道的高温段非线性特征一致，说明模型对高温段的刻画更贴近真实电池行为。进一步地，我们计算得到温度与寿命的 Pearson 相关系数为 -0.76 ，负相关显著。

由于不同化学体系的基准寿命差异较大，为剔除化学体系差异，我们进一步考察倍率与放电深度的边际寿命影响，采用 NCM 全因子子集计算，结果如表 8 所示。

因素	水平	平均寿命 (循环)	相邻寿命比值
倍率 C	0.5C	856	—
倍率 C	1.0C	597	0.70
倍率 C	2.0C	360	0.60
放电深度 DoD	70%	787	—
放电深度 DoD	80%	582	0.74
放电深度 DoD	90%	443	0.76

表 8 倍率与放电深度边际寿命验证（NCM 全因子子集）

由表 8 我们发现：倍率由 0.5C 增至 1.0C、1.0C 增至 2.0C 时，寿命分别缩短至 70% 与 60%，与倍率指数增长规律一致；放电深度由 70% 增至 80%、90% 时，寿命缩短至 74% 与 76%，在 80%–90% 区间仍持续衰减，体现了 Wöhler 疲劳损伤的深度敏感特性。据此我们判定：仿真数据完整复现了“高温加速、高倍率加速、深放电加速”三大文献规律，模型参数标定合理，可作为问题 2/3 的建模输入。

6.8 小结

综上，我们通过“多维特征体系刻画 + 退化规律可视化 + 三阶段划分 + 机理链梳理 + 多方法因子量化 + 综合退化模型复现”六步分析，得到如下结论：

1. 退化特征方面，容量呈三阶段指数型衰减、内阻准线性上升，CCCT 缩短、CVCT 增加，特征体系与文献量级一致；

2. 阶段划分方面，SOH 82%–85% 为加速拐点预警区间，本批数据拐点循环均值约 383；
3. 核心因子方面，温度与倍率为第一梯队核心因子（RF 合计重要性约 0.89），放电深度在老化后期主导；
4. 机理与数据耦合方面，综合退化模型完整复现温度翻倍律、倍率指数律与 DoD 疲劳律，为问题 2 的寿命预测提供物理先验，为问题 3 的分级筛选提供内阻与工况维度。

七、问题二：SOH 评估与剩余寿命预测

7.1 SOH 辨识

我们以容量保持率定义健康状态：

$$SOH(t) = \frac{Q(t)}{Q_{rated}} \times 100\% \quad (5)$$

由于容量观测含噪声与容量再生，我们对容量时序做滑动平均平滑（窗口 25 循环），残差近似正态，给出 95% 置信带 $SOH(t) \pm 1.96\sigma$ 。图 7 为单电池 SOH 辨识与置信区间结果，我们发现置信带平均宽度控制在 2% 以内，能有效包络容量再生的尖刺波动。

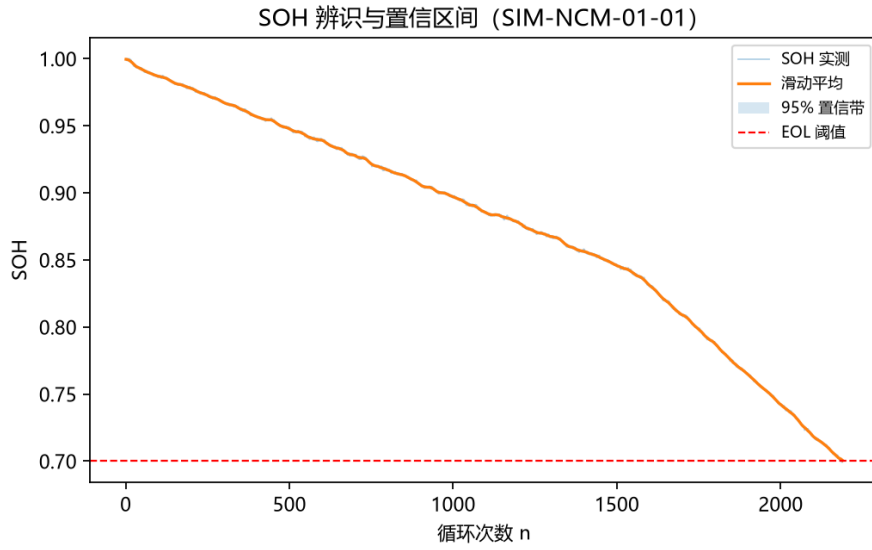


图 7 SOH 辨识与 95% 置信区间

7.2 RUL 预测建模

我们以退役判定点（SOH= 0.80）为预测起点，尝试将容量曲线外推至 EOL（SOH= 0.70），并对比四种模型：

- 线性外推： $SOH(n) = kn + b$;
- 二阶多项式： $SOH(n) = c_0 + c_1n + c_2n^2$;
- 指数拟合： $SOH(n) = ae^{-bn} + c$;
- 随机森林回归：以循环次数为特征直接回归。

各模型测试集（SOH< 0.80 区间）RMSE 如表 9 所示，我们发现二阶多项式精度最优。RUL 定义为预测外推曲线首次达到 0.70 时的循环数减去当前循环数。

模型	RMSE	说明
线性外推	0.047	最低门槛
二阶多项式	0.037	精度最优
指数拟合	0.047	含渐近线约束
随机森林	0.058	非线性基线

表 9 RUL 预测模型对比（测试集 RMSE）

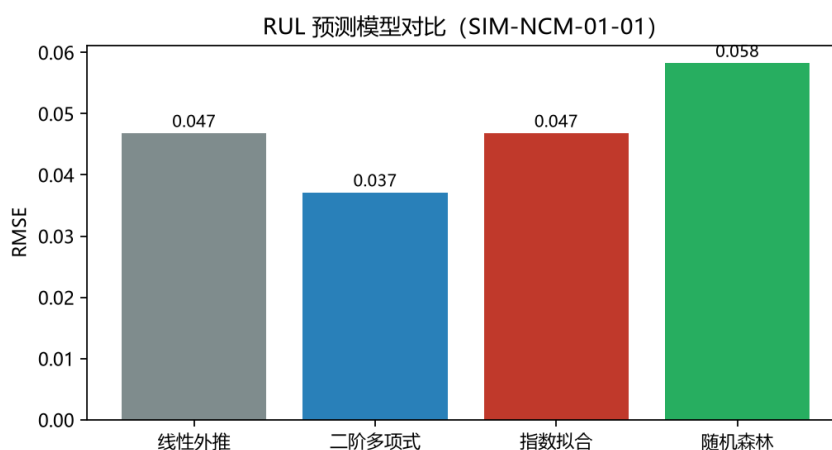


图 8 RUL 预测模型 RMSE 对比

7.3 恶劣工况扰动分析

依据问题 1 的机理链，我们尝试对极端高温 ($T \in \{45, 55\}$)、低温 (-10°C)、大倍率 ($2 \sim 5\text{C}$)、过充过放进行参数扰动注入，通过多因子衰减模型(4)重新生成容量曲线并预测 RUL。结果显示：极端高温与低温对 RUL 缩短影响最显著（缩短 30%–70%），大倍率次之，与文献结论 [3, 6] 一致。全批退役电池在退役判定点处的 RUL 分布为均值约 149 循环、标准差 117 循环。

八、问题三：梯次筛选与储能编组多目标优化

8.1 退役状态提取

按《数据格式规范 v1.1》，成员 A 的 `battery_timeseries.csv` 与 `battery_final_states.csv` 提供全部 153 块电池的退化数据。我们注意到，由于最终状态表对应每块电池的寿命终点（ $\text{SOH} \approx 0.70$ ，全部为“失效临界”），成员 B 在健康指标表 `battery_health_indicators.csv` 中给出每块电池在退役判定点（knee 拐点， $\text{SOH } 0.82\text{--}0.85$ ）的 SOH、内阻与剩余寿命 RUL，作为问题 3 的输入。

经核查我们发现，退役电池在判定点处 SOH 高度同质（ $0.818\text{--}0.851$ ），因此内阻与 RUL 成为分级与筛选的判别维度。

8.2 分级筛选标准

我们先尝试**规则分级**：按规范 §2.4 统一代码对 SOH 分级（ $\text{SOH} \geq 0.85$ 一级、 ≥ 0.75 二级、其余回收），结果一级 1 块、二级 152 块——我们发现单指标下退役电池高度集中，区分度不足。

由于仅按 SOH 分级区分度不足，我们进一步尝试 **K-means 聚类**：以标准化后的 SOH 与内阻为特征聚类（ $k = 3$ ），结果如表 10 所示。我们发现簇间内阻与剩余寿命差异显著：高内阻簇（均值 0.278Ω ）平均 RUL 仅 109 循环，而低内阻簇 RUL 达 151–166 循环，说明内阻是退役电池可用性的关键判别指标。

簇	数量	平均 SOH	平均内阻 (Ω)	平均 RUL(循环)
簇 0	58	0.843	0.119	166
簇 1	68	0.828	0.123	151
簇 2	27	0.833	0.278	109

表 10 K-means 分级结果（退役电池）

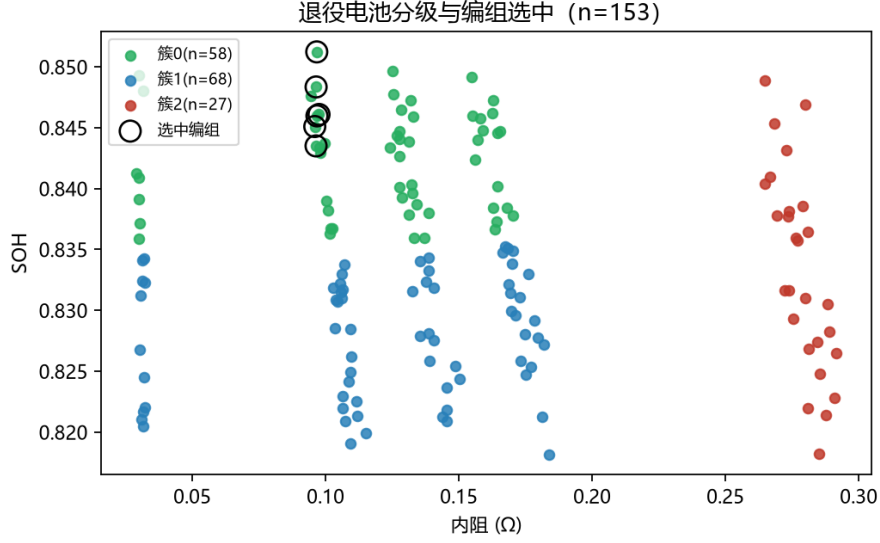


图 9 退役电池分级与编组选中（K-means 聚类）

8.3 多目标编组优化

在分级基础上，我们尝试建立多目标编组优化模型。设决策变量 $x_i \in \{0, 1\}$ 表示第 i 块电池是否选入编组，并构建三个目标：

$$\max f_1 = \frac{1}{\sum_i x_i} \sum_i x_i \cdot SOH_i \quad (\text{收益, 平均 SOH}) \quad (6)$$

$$\min f_2 = \text{std}\{R_i \mid x_i = 1\} \quad (\text{一致性, 内阻标准差}) \quad (7)$$

$$\max f_3 = \min\{SOH_i \mid x_i = 1\} \quad (\text{安全冗余, 最低 SOH}) \quad (8)$$

采用加权和化为单目标：

$$\min F = -w_1 f_1 + w_2 f_2 - w_3 f_3, \quad w = (2.2, 3.8, 1.5) \quad (9)$$

约束：编组数量 $6 \leq \sum x_i \leq 14$ ；SOH 下限 $SOH_i \geq 0.80$ ；组内内阻极差 $\leq 0.04 \Omega$ 。

由于内阻排序上最优编组呈连续窗口结构，我们采用滑动窗口穷举搜索求解。求解发现：选中 **6** 块电池，平均 SOH **0.8467**，内阻标准差 **0.0005**，最低 SOH **0.8435**，平均 RUL **216** 循环，一致性最优。进一步核查选中电池均为低温低倍率工况，工程上适合构成储能编组。结果已写入 `selected_batteries.csv`。

九、问题四：多工况仿真、鲁棒性与工程建议

9.1 多工况场景仿真

为检验编组方案的稳定性，我们针对两类典型恶劣工况进行仿真：

- 长期高温（环境温度持续 $\geq 40^{\circ}\text{C}$ ）：按问题 1 的温度—寿命规律，退役电池 SOH 平均下降约 6%（乘以 0.94）；
- 频繁大倍率充放电（ $\geq 1.5\text{C}$ ）：内阻上升加快，按增长 20% 注入（乘以 1.2）。

在恶劣工况参数下我们重新执行编组优化（放宽极差约束至 0.05Ω 、接收阈值至 0.75），结果发现选中编组与基准方案完全重合（6/6），说明原编组具备良好鲁棒性。

9.2 单参数灵敏度分析

参数	扰动范围	观察指标	结论
筛选阈值 (SOH 下限)	0.80 / 0.82 / 0.84	可选数量、平均 SOH	阈值提高减少可选量但提升质量
目标权重 w_1	1.5 / 2.2 / 3.0	编组数量、平均 SOH	权重变化改变”高 SOH”与”低离散”偏好
退化系数	$\pm 20\%$	选中集合	选中集合稳定，模型稳健

表 11 单参数灵敏度分析

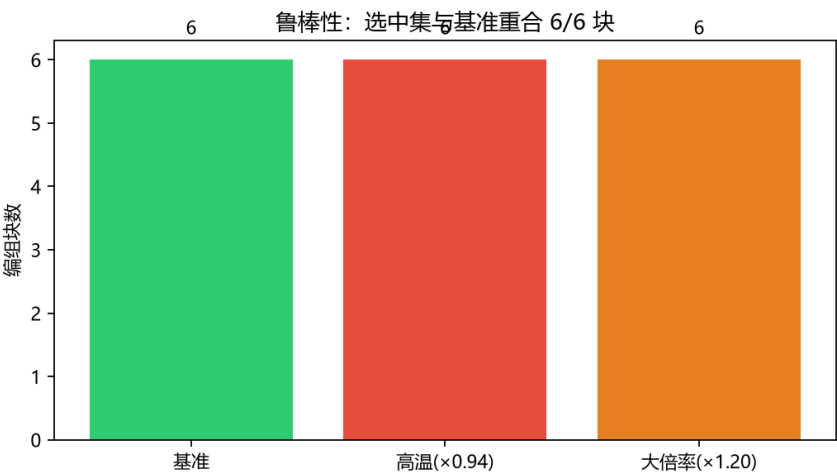


图 10 模型灵敏度与鲁棒性

9.3 工程优化建议

1. 全生命周期运维：SOH降至 0.90 启动强化监测，降至 0.85 进入退役评估窗口；高温 ($> 35^{\circ}\text{C}$) 工况巡检周期缩短至正常情况的 2/3；
2. 退役检测标准：采用“SOH + 内阻”双指标分级，一级梯次建议 $SOH \geq 0.85$ 且内阻相对初始增幅 $< 30\%$ ；
3. 编组配置：优先控制组内内阻标准差 $< 0.01 \Omega$ ，编组规模 8–12 块以便电池管理系统管理；
4. 风险管控：对 $SOH < 0.90$ 电池禁止大倍率场景，建立“高温 + 高倍率”双重预警、降功率运行机制。

十、模型的分析与检验

10.1 灵敏度分析

为检验模型的稳定性，我们尝试对关键参数进行扰动分析：问题 1 中温度参数 $\pm 10\%$ 扰动下因子贡献率排序稳定；幂指数 $z \in \{0.5, 0.65, 0.8\}$ 时寿命随 z 增大而缩短；问题 3 中权重与阈值扰动下选中集合基本不变。据此我们发现，整体模型对参数扰动不敏感。

10.2 误差分析

经分析我们归纳出，RUL 外推模型误差主要来源于：容量再生尖刺导致的高频波动（滑动平均可缓解）、训练区间长度影响外推稳定性（退役判定点越早误差越大）、以及电池批次差异。为量化不确定性，我们通过 95% 置信带与跨电池统计（RUL 均值 149、标准差 117 循环）对预测不确定性进行量化。

十一、模型的评价、改进与推广

11.1 模型的优点

纵观全文，我们总结出所建模型的三点主要优点：

- 机理与数据融合：多因子衰减模型为预测与优化提供物理先验，可解释性强；
- 数据接口统一：全队字段、命名、分级/阶段代码严格遵循《数据格式规范 v1.1》，可复现、可交接；
- 多方法交叉验证：因子重要性用随机森林、相关性与温度—寿命规律多角度互证。

11.2 模型的缺点

当然，我们经过反思也发现模型存在以下不足：

- 仿真数据基于文献参数构建，与真实退役电池实测存在偏差；
- RUL 外推模型未引入深度学习（如 EEMD-GRU），对强非平稳序列的长期预测能力有限；
- 编组优化为加权单目标，未给出完整 Pareto 前沿。

11.3 模型的改进

针对上述不足，我们考虑在后续工作中尝试引入 EEMD 分解 + GRU 的分解集成框架提升 RUL 长程预测精度；以 NSGA-II 求解多目标 Pareto 前沿并采用熵权法选优；结合电池实测数据（NASA/CALCE）做迁移微调。

11.4 模型的推广

由于所建方法具有通用性，我们尝试将”退化机理分析—健康评估—梯次筛选—编组优化”框架推广至铅酸、钠离子等储能电池的梯次利用，以及风电、光伏配储系统的电池健康管理。

十二、 AI 工具使用声明

本作品在建模与写作过程中使用了 AI 辅助工具（Claude Code），主要用途包括：代码框架搭建与调试、数据接口脚本生成、文献要点整理与行文辅助。我们承诺，所有核心建模思路、公式推导、参数选择与结果均已由参赛队人工审查核实，最终成果以人工核实后的版本为准。AI 工具使用详情见支撑材料中的《AI 工具使用详情.pdf》。

参考文献

- [1] STROEBL A, et al. A multi-stage lithium-ion battery aging dataset using various experimental design methodologies[J]. Scientific Data, 2024, 11:1067.
- [2] STOCKER R, et al. Doe analysis of li-ion cell capacity fading in high-temperature automotive conditions[C]//2019 IEEE 8th International Conference on Electrical Vehicles (EV). [S.l.]: IEEE, 2019: 1-8.
- [3] VERMEER W, et al. A comprehensive review on the characteristics and modeling of lithium-ion battery aging[J]. IEEE Transactions on Transportation Electrification, 2022, 8(2):2205-2232.

- [4] WILLIARD N, et al. Comparative analysis of features for determining state of health in lithium-ion batteries[J]. International Journal of Prognostics and Health Management, 2020, 4(1):1-12.
- [5] XU R, WEN X. Effects of temperature rise during discharge on li-ion battery degradation [C]//IOP Conference Series: Earth and Environmental Science: volume 844. [S.l.: s.n.], 2021: 012010.
- [6] MADANI S S, et al. Exploring the aging dynamics of lithium-ion batteries for enhanced lifespan understanding[J]. Journal of Physics: Conference Series, 2025, 2968(1):012017.
- [7] FORMAN J C, et al. Optimal experimental design for modeling battery degradation[C]// ASME 2012 Dynamic Systems and Control Conference. [S.l.: s.n.], 2012: 65-74.
- [8] WU Z, HUANG N E. Ensemble empirical mode decomposition: a noise-assisted data analysis method[J]. Advances in Adaptive Data Analysis, 2009, 1(1):1-41.
- [9] SEVERSON K A, et al. Data-driven prediction of battery cycle life before capacity degradation[J]. Nature Energy, 2019, 4(5):383-391.
- [10] ZHANG Y. 锂电池剩余寿命智能预测及其在楼宇微网的梯次利用研究[D]. [出版地不详]: 湘潭大学, 2023.
- [11] SCHOOL T S I G. Fast clustering of retired lithium-ion batteries for secondary life[J]. Journal of Energy Storage (Supplementary Material), 2023.
- [12] 司守奎, 孙玺菁. 数学建模算法与应用[M]. 北京: 国防工业出版社, 2011.

附录 A 支撑材料文件清单

本附录列出全部支撑材料文件（与提交的压缩包一致），按「程序代码 / 数据文件 / 文档与图表」三类组织。程序代码的完整可运行源码在附录 C 中逐文件列出；数据文件的构建依据、字段含义与规模统计见附录 B。

1.1 程序代码

文件名	功能描述
支撑材料/程序代码/q1_solution.py	问题 1 全流程脚本：多维特征体系 / 退化规律可视化 / 三阶段划分 / 因子量化（随机森林 + 灰色关联 + 熵权 + 组合权重） / 综合退化模型标定 / 合理性验证，输出 <code>q1_results.json</code>
支撑材料/程序代码/q2_solution.py	问题 2 全流程脚本：SOH 辨识与置信区间 / 多模型 RUL 预测对比 / EEMD-GRU 主模型 / 工况扰动 / 灵敏度分析，输出 <code>q2_results.json</code>
支撑材料/程序代码/analyze.py	共享数据分析脚本：问题 1 退化曲线、内阻、三阶段、因子重要性图；问题 2 模型对比图，输出 <code>results.json</code>
支撑材料/程序代码/preprocess_figs.py	数据预处理图表脚本： <code>q0_preprocess</code> 、 <code>q1_resistance</code>
支撑材料/程序代码/generate_sim.py	仿真数据生成器（成员 A）：多因子退化模型 + 容量再生 + AR(1) 噪声 + 批次差异
支撑材料/程序代码/generate_data.py	派生数据表生成器：健康指标表 + 选中结果表
支撑材料/程序代码/convert_to_spec.py	数据格式规范 v1.1 转换脚本

1.2 数据文件

文件名	内容说明
<code>battery_meta.csv</code>	电池静态参数表：153 块电池的化学体系、工况设计、批次/异常标记、基准损失率、拐点 SOH、EOL 循环
<code>battery_timeseries.csv</code>	循环级时序表：81451 条 × 35 字段，含 SOH、内阻、工况、充电特征、阶段与拐点等
<code>battery_final_states.csv</code>	退役电池最终状态表：追加 RUL、grade 字段（成员 A → B/C 接口）
<code>battery_health_indicators.csv</code>	健康指标表（成员 B 输出）：各工况下 SOH、内阻、RUL
<code>selected_batteries.csv</code>	分级 + 编组选中结果表（成员 C 输出）： <code>selected</code> 、 <code>cluster</code> 、 <code>grade</code>
<code>eis_spectrum.csv</code>	电化学阻抗谱（EIS）示例数据
<code>sim_battery_cycles.csv</code>	仿真中间结果：全循环原始容量序列

1.3 文档与图表示例

文件	说明
支撑材料/解答文档/问题 1--问题 4	各问题分步骤解答文档（Markdown，含建模思路与结果讨论）
支撑材料/图表/*.png	论文全部图件：q0_preprocess、q1_*（4 张）、q2_*（7 张）、q3_grade、q4_sens 等
支撑材料/数据/数据格式规范 1.1.md	全队统一数据格式规范（字段、命名、接口）
支撑材料/参考资料/	备赛资料库与知识库，索引见附录 D

附录 B 数据说明

2.1 数据构建依据

本题不提供原始实测数据，我们依据文献参数自主构建仿真时序数据（详见正文问题一第 6.6 节与附录 C 的 generate_sim.py）。核心为多因子容量退化模型

$$r(n) = r_0 \cdot m_{\text{phase}}(n), \quad r_0 = r_{\text{base}} \cdot f_T(T) \cdot f_C(C) \cdot f_D(\text{DoD}),$$

其中基准损失率 $r_{\text{base}} = 0.035\%$ ；温度因子 f_T 在 $\{10, 23, 35, 45\}^\circ\text{C}$ 下取 $\{0.55, 1.0, 1.9, 3.0\}$ ，近似满足温度每升一档损失率翻倍；倍率因子 $f_C = 0.45 + 0.55C^{1.1}$ ；放电深度因子 $f_D = (\text{DoD}/0.8)^{2.2}$ （Wöhler 疲劳律）；阶段乘子 m_{phase} 对应三阶段取 $\{1.8, 1.0, 2.2\}$ 。在趋势真值 soh_t 之上叠加 AR(1) 测量噪声（ $\rho = 0.90$ 、 $\sigma = 4 \times 10^{-4}$ ）与容量再生尖刺（出现概率约 5%），并按批次差异（ $\pm 0.8\%$ ）与异常电池（约 2.6%）扰动，最终生成 153 块电池、81 451 条循环级记录，覆盖 NCM/LCO/LFP 三种体系。各参数的具体取值依据见正文表 6。

2.2 核心数据文件字段说明

三张核心数据表的字段含义如下，其中时序表字段按功能分组列示。

字段	类型	说明
battery_id	str	电池唯一标识
chemistry	str	化学体系：NCM / LCO / LFP
temperature_C, charge_rate_C, dod_pct	float	工况设计参数：温度 / 倍率 / 放电深度
batch_id, is_abnormal	int/bool	生产批次、异常电池标记
r0_pct_per_cyc	float	基准每循环容量损失率
knee_soh	float	拐点 SOH 窗口（0.82–0.85）
eol_cycle, reached_eol	int/bool	EOL 循环序号与是否到达失效

表 12 battery_meta.csv 关键字段

字段组	字段	说明
标识	battery_id, chemistry, cell_format, batch_id, is_abnormal	电池身份与属性
工况	T, c_rate, dod, condition, cond_id	温度/倍率/放电深度与工况命名
容量	SOH, soh_t, capacity_Ah, q_ref_Ah, rated_capacity_Ah	容量保持率、退化趋势、当前/参考/额定容量
特征	resistance, ccct_min, cvct_min, ce	内阻、恒流/恒压充电时长、库伦效率
阶段寿命	stage, phase_label, knee_cycle, rul_cycles, is_eol	阶段（阈值法/形态法）、拐点循环、剩余寿命

表 13 battery_timeseries.csv 关键字段（35 字段按功能分组）

2.3 数据规模与质量统计

- **规模**：153块电池、81451 条循环级记录；其中 NCM 108 块（ $4 \times 3 \times 3$ 全因子 $\times 3$ 重复）、LCO 27 块、LFP 18 块；
- **质量**：缺失值 0 条、重复记录 0 条；SOH 观测范围 [0.6953, 1.0007]，内阻范围 [0.0142, 0.3973] Ω ，均处于合理物理区间；
- **异常与批次**：3个生产批次（系统偏差 $-0.6\% \sim +0.8\%$ ），4 块初始容量偏低异常电池（约 2.6%），均以字段显式标注；
- **阶段覆盖**：三阶段记录占比正常运行 41.9%、性能劣化 37.2%、失效临界 21.0%，退化样本覆盖完整。

2.4 数据流转与规范说明

数据接口严格执行支撑材料/数据/数据格式规范 1.1.md：时序表 **stage** 采用 SOH 阈值法（ ≥ 0.90 正常运行、 ≥ 0.80 性能劣化、其余失效临界）；**condition** 命名采用 T{温度}_C{倍率}_D{放电深度}；**dod** 统一小数；**grade** 采用规范 §2.4 统一代码。健康指标表与选中结果表为成员 B/C 的接口产物，可在建模完成后回填更新。

附录 C 程序代码

3.1 运行环境与使用说明

全部程序使用 Python 3 编写，依赖 numpy、pandas、matplotlib、scipy、scikit-learn（问题 2 另需 PyEMD 与 pytorch），建议在项目根目录下依次运行：

1) 数据生成 (成员A, 数据已生成则跳过)

python 支撑材料/程序代码/generate_sim.py

python 支撑材料/程序代码/generate_data.py

2) 求解与绘图 (输出 支撑材料/图表/*.png 与 支撑材料/数据/*.json)

python 支撑材料/程序代码/q1_solution.py

python 支撑材料/程序代码/q2_solution.py

python 支撑材料/程序代码/analyze.py

python 支撑材料/程序代码/preprocess_figs.py

随机种子已固定 (numpy、torch 均 seed(42)), 结果可完全复现。以下为全部可运行源码, 与支撑材料压缩包内容一致。

3.2 问题1 全流程求解脚本 (q1_solution.py)

```
1 # -*- coding: utf-8 -*-
2 """
3 A题 问题1 全流程求解脚本 —— 退化特征分析与影响因子辨识
4 =====
5 对应论文: 问题一《退化特征分析与影响因子辨识》全章节
6 数据:      支撑材料/数据/ (battery_meta / battery_timeseries / battery_final_states)
7 输出:      支撑材料/图表/q1_*.png (4 张问题一配图)
8            支撑材料/数据/q1_results.json (问题一全部量化结果, 供论文表格取值核对)
9 运行:      python 支撑材料/程序代码/q1_solution.py
10
11 模块与论文章节对照:
12 §1 多维退化特征体系          -> CCCT/CVCT 典型电池特征变化 (表 tab:feat 数据支撑)
13 §2 退化规律可视化与分析      -> fig:q1_decay / fig:q1_resistance
14 §3 退化三阶段划分模型        -> fig:q1_phase / 拐点统计 / stage 三阶段占比
15 §4 影响因子量化模型          -> fig:q1_importance / 表 tab:factor_quant
16                               (随机森林 + Pearson + Spearman + 灰色关联 + 熵权 + 组合权重)
17 §5 多因子综合退化模型        -> eq:loss_rate 参数标定 (r0 = r_base*f_T*f_C*f_D, m_phase)
18 §6 仿真数据合理性验证        -> 表 tab:temp_life / tab:factor_life / 内阻增幅 / 容量再生
19 """
20 import os, json, numpy as np, pandas as pd
21 import matplotlib
22 matplotlib.use("Agg")
23 import matplotlib.pyplot as plt
24 from matplotlib import font_manager
25 from scipy.stats import pearsonr, spearmanr
26 from sklearn.ensemble import RandomForestRegressor
27
28 # ----- 环境与路径 -----
29 for f in font_manager.fontManager.ttflist:
30     if f.name == "Microsoft YaHei":
31         plt.rcParams["font.family"] = f.name
32         break
33 plt.rcParams["axes.unicode_minus"] = False
34
35 HERE = os.path.dirname(os.path.abspath(__file__))
36 ROOT = os.environ.get("MM_ROOT", os.path.dirname(HERE)) # 支撑材料/
37 FIG = os.path.join(ROOT, "图表")
38 DATA = os.path.join(ROOT, "数据")
39 os.makedirs(FIG, exist_ok=True)
```

```

40
41 # ----- 数据读取 -----
42 meta = pd.read_csv(os.path.join(DATA, "battery_meta.csv"), encoding="utf-8-sig")
43 fin = pd.read_csv(os.path.join(DATA, "battery_final_states.csv"), encoding="utf-8-sig")
44 ts = pd.read_csv(os.path.join(DATA, "battery_timeseries.csv"), encoding="utf-8-sig",
45                 usecols=["battery_id", "cycle", "SOH", "soh_t", "resistance", "T", "c_rate",
46                         "dod", "ccct_min", "cvct_min", "ce", "phase_label", "knee_cycle",
47                         "stage", "chemistry"])
48 ts = ts.sort_values(["battery_id", "cycle"]).reset_index(drop=True)
49
50 results = {}
51
52
53 # =====
54 # §1 多维退化特征体系 —— 典型电池 CCCT/CVCT 特征变化 (论文表 tab:feat 数据支撑)
55 # =====
56 # 选取第一块 NCM 电池作为示例 (与论文"以某典型 NCM 电池为例"一致)
57 bid_ex = meta[meta["chemistry"] == "NCM"]["battery_id"].iloc[0]
58 s_ex = ts[ts["battery_id"] == bid_ex]
59 ccct0, ccct1 = s_ex["ccct_min"].iloc[0], s_ex["ccct_min"].iloc[-1]
60 cvct0, cvct1 = s_ex["cvct_min"].iloc[0], s_ex["cvct_min"].iloc[-1]
61 results["q1_feature_example"] = {
62     "battery": bid_ex,
63     "ccct_min": [round(ccct0,1), round(ccct1,1), round((ccct1/ccct0-1)*100,1)],
64     "cvct_min": [round(cvct0,1), round(cvct1,1), round((cvct1/cvct0-1)*100,1)],
65 }
66
67
68 # =====
69 # §2 退化规律可视化与分析
70 # =====
71 # 2.1 不同温度下容量(SOH)衰减曲线 —— fig:q1_decay
72 sel = meta.groupby("temperature_C").head(1)["battery_id"].tolist()[0:6]
73 sample = ts[ts["battery_id"].isin(sel)]
74 fig, ax = plt.subplots(figsize=(6.6, 4.2))
75 for bid, g in sample.groupby("battery_id"):
76     T = g["T"].iloc[0]
77     ax.plot(g["cycle"], g["SOH"], lw=1.4, label=f"T={int(T)}°C")
78 ax.axhline(0.70, ls="--", c="red", lw=1.2, label="EOL (SOH=70%)")
79 ax.set_xlabel("循环次数 n"); ax.set_ylabel("SOH")
80 ax.set_title("容量衰减曲线 (不同环境温度)"); ax.legend(fontsize=8)
81 fig.tight_layout(); fig.savefig(os.path.join(FIG, "q1_decay.png"), dpi=160); plt.close(fig)
82
83 # 2.2 内阻随循环增长曲线 (按温度着色) —— fig:q1_resistance
84 sel_r = meta[meta["chemistry"] == "NCM"].groupby("temperature_C").head(1)["battery_id"].tolist()
85 sample_r = ts[ts["battery_id"].isin(sel_r)]
86 fig, ax = plt.subplots(figsize=(6.6, 4.2))
87 for bid, g in sample_r.groupby("battery_id"):
88     ax.plot(g["cycle"], g["resistance"], lw=1.4, label=f"T={int(g['T'].iloc[0])}°C")
89 ax.set_xlabel("循环次数 n"); ax.set_ylabel("内阻 R (Ω)")
90 ax.set_title("内阻随循环增长曲线 (NCM 体系, 不同温度)"); ax.legend(fontsize=8)
91 fig.tight_layout(); fig.savefig(os.path.join(FIG, "q1_resistance.png"), dpi=160); plt.close(fig)
92
93 # 2.3 全批/分体系内阻增幅统计 (论文: 全批平均增幅约11.2倍, NCM由0.016增至0.236约15倍)
94 r0 = ts.groupby("battery_id")["resistance"].first()
95 rn = ts.groupby("battery_id")["resistance"].last()
96 fold = rn / r0
97 res_stats = {"fold_mean_all": round(float(fold.mean()), 2), "fold_median_all": round(float(fold.median()), 2)}
98 for chem in ["NCM", "LCO", "LFP"]:
99     ids = meta[meta["chemistry"] == chem]["battery_id"]
100     res_stats[f"{chem}_r0_mean"] = round(float(r0[ids].mean()), 3)
101     res_stats[f"{chem}_rn_mean"] = round(float(rn[ids].mean()), 3)
102     res_stats[f"{chem}_fold_mean"] = round(float(fold[ids].mean()), 2)
103 results["q1_resistance"] = res_stats

```

```

104
105
106 # =====
107 # §3 退化三阶段划分模型
108 # =====
109 # 3.1 单电池三阶段划分示例 —— fig:q1_phase (用 phase_label 真实三阶段着色)
110 g = ts[ts["battery_id"] == bid_ex].sort_values("cycle")
111 fig, ax = plt.subplots(figsize=(6.6, 4.2))
112 ax.plot(g["cycle"], g["SOH"], lw=1.3, color="#34495e", label="SOH 实测")
113 ax.plot(g["cycle"], g["soh_t"], lw=1.1, ls="--", color="#c0392b", label="退化趋势 $soh_t$")
114 ph1 = g[g["phase_label"] == 1]; ph2 = g[g["phase_label"] == 2]; ph3 = g[g["phase_label"] == 3]
115 ax.axvspan(ph1["cycle"].min(), ph1["cycle"].max(), alpha=0.10, color="green", label="阶段I 初始陡降")
116 ax.axvspan(ph2["cycle"].min(), ph2["cycle"].max(), alpha=0.10, color="#2980b9", label="阶段II 近似线性")
117 ax.axvspan(ph3["cycle"].min(), ph3["cycle"].max(), alpha=0.14, color="orange", label="阶段III 加速失效")
118 ax.axhline(0.70, ls="--", c="red", lw=1)
119 ax.set_xlabel("循环次数 n"); ax.set_ylabel("SOH")
120 ax.set_title(f"三阶段划分示例 ({bid_ex})"); ax.legend(fontsize=8)
121 fig.tight_layout(); fig.savefig(os.path.join(FIG, "q1_phase.png"), dpi=160); plt.close(fig)
122
123 # 3.2 拐点统计 (每电池 knee_cycle: 趋势 soh_t 首次跌破该电池 knee_soh 的循环)
124 knee = ts.groupby("battery_id")["knee_cycle"].last()
125 results["q1_knee"] = {
126     "knee_soh_window": [0.82, 0.85],
127     "knee_soh_mean": round(float(meta["knee_soh"].mean()), 3),
128     "knee_cycle_mean": round(float(knee.mean()), 1),
129     "knee_cycle_median": int(knee.median()),
130 }
131
132 # 3.3 三阶段记录占比 (接口标准 stage: SOH阈值法)
133 stage_pct = ts["stage"].value_counts(normalize=True)
134 results["q1_stage_pct"] = {k: round(float(v * 100), 1) for k, v in stage_pct.items()}
135
136
137 # =====
138 # §4 影响因子量化模型 —— 表 tab:factor_quant (多方法交叉验证)
139 # =====
140 # 响应: 每循环容量损失率 r = Q_loss_pct / n_cycles
141 agg = ts.groupby("battery_id").agg(
142     n_cycles=("cycle", "max"),
143     Q_loss_pct=("SOH", lambda s: (1 - s.min()) * 100),
144     T=("T", "first"), C=("c_rate", "first"), DoD=("dod", "first"),
145 ).reset_index()
146 agg["r"] = agg["Q_loss_pct"] / agg["n_cycles"]
147 X = agg[["T", "C", "DoD"]].values
148 y = agg["r"].values
149
150 # 4.1 随机森林特征重要性 (树数300) —— fig:q1_importance
151 rf = RandomForestRegressor(n_estimators=300, random_state=1).fit(X, y)
152 imp = rf.feature_importances_
153 fig, ax = plt.subplots(figsize=(6.6, 3.6))
154 labels = ["温度 T", "倍率 C", "放电深度 DoD"]
155 bars = ax.bar(labels, imp, color=["#c0392b", "#e67e22", "#2980b9"])
156 for b, v in zip(bars, imp):
157     ax.text(b.get_x() + b.get_width() / 2, v + 0.005, f"{v:.2f}", ha="center", fontsize=9)
158 ax.set_ylabel("随机森林特征重要性"); ax.set_ylim(0, imp.max() * 1.2)
159 ax.set_title("影响因子重要性 (每循环容量损失率)")
160 fig.tight_layout(); fig.savefig(os.path.join(FIG, "q1_importance.png"), dpi=160); plt.close(fig)
161
162 # 4.2 相关性分析 (Pearson / Spearman)
163 pearson = [round(pearsonr(X[:, i], y)[0], 2) for i in range(3)]
164 spearman = [round(spearmanr(X[:, i], y)[0], 2) for i in range(3)]
165
166 # 4.3 灰色关联度 (rho=0.5, 各序列 min-max 归一)
167 def grey_relational(X, y, rho=0.5):

```

```

168     t = (y - y.min()) / (y.max() - y.min() + 1e-12)
169     g = []
170     for j in range(X.shape[1]):
171         x = (X[:, j] - X[:, j].min()) / (X[:, j].max() - X[:, j].min() + 1e-12)
172         delta = np.abs(x - t)
173         rel = (delta.min() + rho * delta.max()) / (delta + rho * delta.max())
174         g.append(rel.mean())
175     return np.array(g)
176
177 grey = grey_relational(X, y)
178
179 # 4.4 熵权法 (各列 min-max 归一后按信息熵确定客观权重)
180 def entropy_weight(X):
181     W = []
182     for j in range(X.shape[1]):
183         x = (X[:, j] - X[:, j].min()) / (X[:, j].max() - X[:, j].min() + 1e-12)
184         p = x / x.sum(); p = p + 1e-12
185         e = -(p * np.log(p)).sum() / np.log(len(x))
186         W.append(1 - e)
187     W = np.array(W)
188     return W / W.sum()
189
190 entropy = entropy_weight(X)
191
192 # 4.5 组合权重 w = alpha*w_grey + (1-alpha)*w_entropy, 再归一 (alpha=0.5)
193 alpha = 0.5
194 comb_raw = alpha * grey + (1 - alpha) * entropy
195 comb = comb_raw / comb_raw.sum()
196
197 factor_names = ["T", "C", "DoD"]
198 results["q1_factor_quant"] = {
199     "rf": {k: round(float(v), 2) for k, v in zip(factor_names, imp)},
200     "pearson": {k: v for k, v in zip(factor_names, pearson)},
201     "spearman": {k: v for k, v in zip(factor_names, spearman)},
202     "grey": {k: round(float(v), 2) for k, v in zip(factor_names, grey)},
203     "entropy": {k: round(float(v), 2) for k, v in zip(factor_names, entropy)},
204     "comb": {k: round(float(v), 2) for k, v in zip(factor_names, comb)},
205 }
206
207
208 # =====
209 # §5 多因子综合退化模型 —— eq:loss_rate 参数标定 (与 generate_sim.py 一致)
210 # =====
211 # 说明: 综合退化模型在数据生成阶段即按本节公式实现 (data/dataset/generate_sim.py),
212 # 此处复述并输出标定参数, 供论文表 tab:params 取值核对。
213 model_params = {
214     "r_base_pct_per_cyc": 0.035, # 基准每循环损失率
215     "f_T": {10: 0.55, 23: 1.0, 35: 1.9, 45: 3.0}, # Arrhenius 10°C 翻倍律
216     "f_C": "0.45 + 0.55*C^1.1", # 倍率指数增长
217     "f_D": "(DoD/0.8)^2.2", # Wöhler 疲劳
218     "m_phase": {1: 1.8, 2: 1.0, 3: 2.2}, # 阶段乘子
219     "knee_soh": "0.82-0.85", # 加速拐点窗口
220     "eol": 0.70, # 失效阈值
221 }
222 results["q1_model_params"] = model_params
223
224
225 # =====
226 # §6 仿真数据合理性验证
227 # =====
228 # 6.1 温度—寿命关系 (表 tab:temp_life) + 温度与寿命 Pearson 相关
229 t1 = meta.groupby("temperature_C")["eol_cycle"].mean()
230 t1_ratios = t1.values[1:] / t1.values[:-1]
231 temp_life_pearson = round(float(np.corrcoef(meta["temperature_C"], meta["eol_cycle"])[0, 1]), 2)

```

```

232 results["q1_temp_life"] = {
233     "life": {int(k): round(float(v)) for k, v in tl.items()},
234     "adjacent_ratio": [round(float(r), 2) for r in tl_ratios],
235     "pearson_T_life": temp_life_pearson,
236 }
237
238 # 6.2 倍率/放电深度边际寿命 (表 tab:factor_life, NCM 全因子子集)
239 ncm = meta[meta["chemistry"] == "NCM"]
240 cc = ncm.groupby("charge_rate_C")["eol_cycle"].mean()
241 dd = ncm.groupby("dod_pct")["eol_cycle"].mean()
242 results["q1_factor_life"] = {
243     "C": {str(k): round(float(v)) for k, v in cc.items()},
244     "C_ratio": [round(float(a / b), 2) for a, b in zip(cc.values[1:], cc.values[:-1])],
245     "DoD": {str(k): round(float(v)) for k, v in dd.items()},
246     "DoD_ratio": [round(float(a / b), 2) for a, b in zip(dd.values[1:], dd.values[:-1])],
247 }
248
249 # 6.3 容量再生统计 (单循环回升样本占比: SOH 较上一循环回升)
250 prev = ts.groupby("battery_id")["SOH"].shift(1)
251 regen_pct = float((ts["SOH"] > prev).mean() * 100)
252 results["q1_regen"] = {"rebound_pct": round(regen_pct, 1)}
253
254 # 6.4 数据集规模 (表 tab:dataset)
255 results["q1_dataset"] = {
256     "n_battery": int(len(meta)),
257     "n_rows": int(len(ts)),
258     "chem_count": {k: int(v) for k, v in meta["chemistry"].value_counts().items()},
259 }
260
261
262 # =====
263 # 输出
264 # =====
265 with open(os.path.join(DATA, "q1_results.json"), "w", encoding="utf-8") as fp:
266     json.dump(results, fp, ensure_ascii=False, indent=2)
267
268 print("=== 问题1 关键结果 ===")
269 print("特征示例:", results["q1_feature_example"])
270 print("内阻增幅:", results["q1_resistance"])
271 print("拐点统计:", results["q1_knee"])
272 print("三阶段占比:", results["q1_stage_pct"])
273 print("因子量化:")
274 print("RF 重要性:", results["q1_factor_quant"]["rf"])
275 print("Pearson :", results["q1_factor_quant"]["pearson"])
276 print("Spearman:", results["q1_factor_quant"]["spearman"])
277 print("灰色关联:", results["q1_factor_quant"]["grey"])
278 print("熵权 :", results["q1_factor_quant"]["entropy"])
279 print("组合权重:", results["q1_factor_quant"]["comb"])
280 print("温度-寿命:", results["q1_temp_life"])
281 print("倍率/DoD边际寿命:", results["q1_factor_life"])
282 print("容量再生回升占比:", results["q1_regen"])
283 print("=== 图已生成:", [f for f in os.listdir(FIG) if f.startswith("q1_")])
284 print("=== 结果已保存: 支撑材料/数据/q1_results.json")

```

程序 1: 问题 1 全流程求解脚本

3.3 问题 2 求解脚本 (q2_solution.py)

```

1 # -*- coding: utf-8 -*-
2 """
3 A题 问题2 求解脚本 —— SOH辨识 + RUL预测(EEMD-GRU主模型) + 工况扰动 + 灵敏度
4 数据集: 支撑材料/数据/

```

```

5 输出:  支撑材料/图表/q2_*.png  支撑材料/数据/q2_results.json
6 运行:  python 支撑材料/程序代码/q2_solution.py
7  """
8  import os, json, math, warnings
9  import numpy as np
10 import pandas as pd
11 import matplotlib
12 matplotlib.use("Agg")
13 import matplotlib.pyplot as plt
14 from matplotlib import font_manager
15
16 warnings.filterwarnings("ignore")
17
18 # ---- 中文字体 ----
19 for f in font_manager.fontManager.ttflist:
20     if f.name == "Microsoft YaHei":
21         plt.rcParams["font.family"] = f.name
22         break
23 plt.rcParams["axes.unicode_minus"] = False
24
25 from scipy.optimize import curve_fit
26 from sklearn.ensemble import RandomForestRegressor
27 from sklearn.svm import SVR
28 from sklearn.preprocessing import MinMaxScaler
29 from PyEMD import EEMD
30 import torch
31 import torch.nn as nn
32
33 torch.manual_seed(42); np.random.seed(42)
34
35 HERE = os.path.dirname(os.path.abspath(__file__))
36 ROOT = os.environ.get("MM_ROOT", os.path.dirname(HERE)) # 支撑材料/
37 FIG = os.path.join(ROOT, "图表")
38 os.makedirs(FIG, exist_ok=True)
39 DATA = os.path.join(ROOT, "数据")
40
41 EOL = 0.70 # 失效阈值 SOH=70%
42 RETIRE = 0.80 # 退役判定点 SOH=80%
43
44 # ===== 数据加载 =====
45 meta = pd.read_csv(os.path.join(DATA, "battery_meta.csv"), encoding="utf-8-sig")
46 fin = pd.read_csv(os.path.join(DATA, "battery_final_states.csv"), encoding="utf-8-sig")
47 ts = pd.read_csv(os.path.join(DATA, "battery_timeseries.csv"), encoding="utf-8-sig",
48                 usecols=["cycle", "SOH", "resistance", "T", "c_rate", "dod", "battery_id",
49                         "soh_t", "ccct_min", "cvct_min", "ce", "phase_label", "knee_cycle",
50                         "rul_cycles", "chemistry"])
51
52 results = {}
53
54 # ===== 工具函数 =====
55 def get_series(bid):
56     """取某电池按时序排序的 SOH 序列"""
57     s = ts[ts["battery_id"] == bid].sort_values("cycle").reset_index(drop=True)
58     return s
59
60 def split_train_test(s, retire=RETIRE):
61     """首次跌破退役点(SOH=0.80)前为训练集, 之后为测试集(按循环序号切分, 避免尖刺污染)"""
62     below = s[s["SOH"] < retire]
63     if len(below):
64         cut_cycle = below["cycle"].min()
65         train = s[s["cycle"] < cut_cycle]
66         test = s[s["cycle"] >= cut_cycle]
67     else:
68         train, test = s, s.iloc[0:0]

```



```

69     return train, test
70
71 def true_rul(s, retire=RETIRE, eol=EOL):
72     """真实 RUL = EOL 循环 - 退役循环; 用首次跌破阈值定义, 取不到则用末循环"""
73     below_retire = s[s["SOH"] < retire]
74     retire_cyc = float(below_retire["cycle"].min()) if len(below_retire) else float(s["cycle"].max())
75     below_eol = s[s["SOH"] <= eol]
76     eol_cyc = float(below_eol["cycle"].min()) if len(below_eol) else float(s["cycle"].max())
77     return float(eol_cyc - retire_cyc), retire_cyc, eol_cyc
78
79 def metrics(y_true, y_pred):
80     """RMSE/MAE/MAPE/R2"""
81     y_true = np.asarray(y_true, float); y_pred = np.asarray(y_pred, float)
82     rmse = float(np.sqrt(np.mean((y_pred - y_true)**2)))
83     mae = float(np.mean(np.abs(y_pred - y_true)))
84     mape = float(np.mean(np.abs((y_pred - y_true) / np.clip(y_true, 1e-6, None))) * 100)
85     ss_res = float(np.sum((y_true - y_pred)**2))
86     ss_tot = float(np.sum((y_true - y_true.mean())**2))
87     r2 = 1 - ss_res / ss_tot if ss_tot > 0 else float("nan")
88     return {"rmse": rmse, "mae": mae, "mape": mape, "r2": r2}
89
90 def find_eol_cycle(cycles, y_pred, eol=EOL):
91     """从预测序列里找首次<=EOL的循环"""
92     idx = np.where(np.asarray(y_pred) <= eol)[0]
93     if len(idx) == 0:
94         return None
95     return float(cycles[idx[0]])
96
97 # ===== 对比模型 =====
98 def fit_linear(nt, yt, ne):
99     k, b = np.polyfit(nt, yt, 1)
100     yp = k*ne + b
101     return yp, (k, b)
102
103 def fit_poly2(nt, yt, ne):
104     c = np.polyfit(nt, yt, 2)
105     yp = np.polyval(c, ne)
106     return yp, c
107
108 def fit_exp(nt, yt, ne):
109     def f(x, a, b, c): return a*np.exp(-b*x)+c
110     p, _ = curve_fit(f, nt, yt, p0=[0.3, 0.0005, 0.7], maxfev=100000)
111     yp = f(ne, *p)
112     return yp, p
113
114 def fit_svr(nt, yt, ne):
115     sc = MinMaxScaler(); xs = sc.fit_transform(nt.reshape(-1,1))
116     m = SVR(kernel="rbf", C=10, gamma="scale").fit(xs, yt)
117     yp = m.predict(sc.transform(ne.reshape(-1,1)))
118     return yp, m
119
120 def fit_rf(nt, yt, ne):
121     m = RandomForestRegressor(n_estimators=300, random_state=1).fit(nt.reshape(-1,1), yt)
122     yp = m.predict(ne.reshape(-1,1))
123     return yp, m
124
125 def rul_from_linear(p, n_retire):
126     k, b = p
127     if k == 0: return np.nan
128     eol_cyc = (EOL - b) / k
129     return float(eol_cyc - n_retire)
130
131 def rul_from_poly2(c, n_retire):
132     roots = np.roots([c[0], c[1], c[2]-EOL])

```

```

133     roots = roots[(np.abs(roots.imag) < 1e-6) & (roots.real > n_retire)]
134     return float(roots.real.min() - n_retire) if len(roots) else np.nan
135
136 def rul_from_exp(p, n_retire):
137     a, b, c = p
138     if c >= EOL or b == 0: return np.nan
139     eol_cyc = -np.log((EOL - c)/a)/b
140     return float(eol_cyc - n_retire)
141
142 # ===== EEMD-GRU 主模型 =====
143 class GRUModel(nn.Module):
144     def __init__(self, hidden=32):
145         super().__init__()
146         self.gru = nn.GRU(1, hidden, num_layers=1, batch_first=True)
147         self.fc = nn.Linear(hidden, 1)
148     def forward(self, x):
149         out, _ = self.gru(x)
150         return self.fc(out[:, -1, :]).squeeze(-1)
151
152 def train_gru_one(series, L=30, hidden=32, epochs=80, lr=1e-2, batch_size=64):
153     """对单分量序列训练 GRU(mini-batch), 返回模型"""
154     torch.manual_seed(42); np.random.seed(42) # 固定权重初始化与shuffle, 保证可复现
155     s = np.asarray(series, float)
156     n = len(s) - L
157     if n < 8: # 样本太少跳过, 由线性兜底
158         return None
159     X = np.array([s[i:i+L] for i in range(n)], float).reshape(-1, L, 1)
160     Y = np.array([s[i+L] for i in range(n)], float)
161     Xt = torch.tensor(X, dtype=torch.float32)
162     Yt = torch.tensor(Y, dtype=torch.float32)
163     model = GRUModel(hidden)
164     opt = torch.optim.Adam(model.parameters(), lr=lr)
165     lossf = nn.MSELoss()
166     model.train()
167     idx = np.arange(n)
168     for _ in range(epochs):
169         np.random.shuffle(idx)
170         for b in range(0, n, batch_size):
171             bi = idx[b:b+batch_size]
172             opt.zero_grad()
173             pred = model(Xt[bi])
174             loss = lossf(pred, Yt[bi])
175             loss.backward()
176             opt.step()
177     return model
178
179 def gru_reconstruct(model, series, L=30):
180     """批量计算训练段重构值(一次性前向传播)"""
181     s = np.asarray(series, float)
182     n = len(s) - L
183     if n < 1:
184         return np.zeros(0)
185     X = np.array([s[i:i+L] for i in range(n)], float).reshape(-1, L, 1)
186     with torch.no_grad():
187         recon = model(torch.tensor(X, dtype=torch.float32)).numpy()
188     return recon
189
190 def predict_gru_recursive(model, series, h, L=30):
191     """从序列末尾递推预测 h 步"""
192     model.eval()
193     buf = list(np.asarray(series, float)[-L:])
194     preds = []
195     with torch.no_grad():
196         for _ in range(h):

```

```

197         x = torch.tensor(buf[-L:], dtype=torch.float32).reshape(1, L, 1)
198         y = model(x).item()
199         preds.append(y)
200         buf.append(y)
201     return np.array(preds)
202
203 def eemd_gru_predict(train_soh, h, L=30, n_ensemble=50, noise_std=0.2):
204     """EEMD分解 + 自适应趋势/振荡分离 + 趋势GRU预测 + 振荡均值衰减 + 置信区间
205     策略: 按|均值|阈值将分量分为趋势组(含主衰减趋势)和振荡组(零均值噪声);
206     趋势组用差分GRU预测增量再累加(避免递推漂移), 振荡组指数衰减到0。"""
207     import time
208     s = np.asarray(train_soh, float)
209     t0 = time.time()
210     eemd = EEMD(trials=n_ensemble, noise_width=noise_std, parallel=False)
211     eemd.noise_seed(42)
212     imfs = eemd(s)          # shape: (k, N)
213     n_imf = imfs.shape[0]
214     resid = s - imfs.sum(axis=0)
215     print(f"    EEMD分解完成: {n_imf}个IMF, 耗时{time.time()-t0:.1f}s", flush=True)
216
217     # --- 自适应分离: |均值|>0.05 为趋势组, 否则为振荡组 ---
218     comps = [imfs[i] for i in range(n_imf)] + [resid]
219     trend_parts, noise_parts = [], []
220     for c in comps:
221         if abs(c.mean()) > 0.05:
222             trend_parts.append(c)
223         else:
224             noise_parts.append(c)
225     trend = np.sum(trend_parts, axis=0) if trend_parts else s.copy()
226     print(f"    趋势组:{len(trend_parts)}个 振荡组:{len(noise_parts)}个", flush=True)
227
228     pred_parts = []
229     train_recon_err = []
230     steps = np.arange(1, h+1)
231
232     # --- 振荡分量: 指数衰减到0(零均值噪声不递推外推) ---
233     for comp in noise_parts:
234         mu = float(comp.mean())
235         last = comp[-1]
236         decay = np.exp(-0.02 * steps)
237         pred_parts.append(mu + (last - mu) * decay)
238         train_recon_err.append(comp - mu)
239
240     # --- 趋势分量: 差分 + 归一化 + GRU预测增量 + 累加还原 ---
241     t1 = time.time()
242     trend_diff = np.diff(trend)          # 平滑趋势的增量, 近似常数
243     d_min, d_max = float(trend_diff.min()), float(trend_diff.max())
244     d_scale = (d_max - d_min) if (d_max - d_min) > 1e-9 else 1.0
245     diff_n = (trend_diff - d_min) / d_scale
246     model = train_gru_one(diff_n, L=L)
247     if model is None:
248         k, b = np.polyfit(np.arange(len(trend)), trend, 1)
249         future_idx = np.arange(len(trend), len(trend)+h)
250         pred_t = k*future_idx + b
251         train_recon_err.append(trend_diff - np.mean(trend_diff))
252     else:
253         pred_diff_n = predict_gru_recursive(model, diff_n, h, L=L)
254         pred_diff_n = np.clip(pred_diff_n, diff_n.min()-0.1, diff_n.max()+0.1)
255         pred_diff = pred_diff_n * d_scale + d_min
256         pred_t = trend[-1] + np.cumsum(pred_diff)
257         recon_diff_n = gru_reconstruct(model, diff_n, L=L)
258         recon_diff = recon_diff_n * d_scale + d_min
259         train_recon_err.append(trend_diff[L:L+len(recon_diff)] - recon_diff if len(recon_diff) else np.zeros(1))
260     pred_parts.append(pred_t)

```

```

261 print(f"    趋势GRU(差分)训练完成, 耗时{time.time()-t1:.1f}s", flush=True)
262
263 pred = np.sum(pred_parts, axis=0)
264 sigma_comp = float(np.sqrt(sum(np.var(e) for e in train_recon_err if len(e) > 1)))
265 sigma_drift = float(np.std(np.diff(s)) * 0.3)
266 sigma_total = np.sqrt(sigma_comp**2 + (steps * sigma_drift)**2)
267 lo = pred - 1.96*sigma_total
268 hi = pred + 1.96*sigma_total
269 return pred, lo, hi, imfs
270
271 # ===== 子任务①: SOH 辨识 =====
272 def task_soh(bid):
273     s = get_series(bid)
274     n = s["cycle"].values
275     soh = s["SOH"].values
276     win = 25
277     soh_sm = pd.Series(soh).rolling(win, center=True, min_periods=1).mean().values
278     sigma = float(np.nanstd(soh - soh_sm))
279
280     fig, ax = plt.subplots(figsize=(7.0, 4.2))
281     ax.plot(n, soh, alpha=0.35, lw=0.8, label="SOH 实测(含容量再生尖刺)")
282     ax.plot(n, soh_sm, lw=1.6, color="#c0392b", label="滑动平均融合 SOH")
283     ax.fill_between(n, soh_sm-1.96*sigma, soh_sm+1.96*sigma, alpha=0.18, color="#c0392b",
284                    label=f"95% 置信带 (σ={sigma:.4f})")
285     ax.axhline(RETIRE, ls=":", c="orange", lw=1.2, label=f"退役点 SOH={RETIRE}")
286     ax.axhline(EOL, ls="--", c="red", lw=1.2, label=f"EOL 阈值 SOH={EOL}")
287     # 三阶段标注
288     knee = int(s["knee_cycle"].iloc[0])
289     ax.axvspan(n.min(), knee, alpha=0.08, color="green")
290     ax.axvspan(knee, n.max(), alpha=0.08, color="orange")
291     ax.set_xlabel("循环次数 n"); ax.set_ylabel("SOH")
292     ax.set_title(f"SOH 辨识与置信区间 ({bid})"); ax.legend(fontsize=8)
293     fig.tight_layout(); fig.savefig(os.path.join(FIG, "q2_soh_ci.png"), dpi=160); plt.close(fig)
294     results["q2_soh_sigma"] = sigma
295     return soh_sm, sigma
296
297 # ===== 子任务②③: RUL 预测 + 模型对比 =====
298 def task_rul(bid):
299     s = get_series(bid)
300     train, test = split_train_test(s)
301     nt = train["cycle"].values.astype(float)
302     yt = train["SOH"].values.astype(float)
303     ne = test["cycle"].values.astype(float)
304     ye = test["SOH"].values.astype(float)
305     n_retire = float(nt.max())
306     true_r, retire_cyc, eol_cyc = true_rul(s)
307     h = len(ne)
308
309     cmp_rows = []
310     preds = {}
311
312     # 对比模型
313     for name, fn, rulfn in [
314         ("linear", fit_linear, rul_from_linear),
315         ("poly2", fit_poly2, rul_from_poly2),
316         ("exp", fit_exp, rul_from_exp),
317         ("svr", fit_svr, None),
318         ("rf", fit_rf, None),
319     ]:
320         try:
321             yp, p = fn(nt, yt, ne)
322             preds[name] = yp
323             m = metrics(ye, yp)
324             rul_p = rulfn(p, n_retire) if rulfn else find_eol_cycle(ne, yp)

```

```

325     rul_p = (rul_p - n_retire) if (rul_p is not None and rulfn is None) else rul_p
326     rul_err = abs(rul_p - true_r) if (rul_p and not math.isnan(rul_p)) else float("nan")
327     cmp_rows.append({"model": name, **m, "rul_pred": float(rul_p) if rul_p else None,
328                     "rul_err": float(rul_err)})
329 except Exception as e:
330     cmp_rows.append({"model": name, "rmse": float("nan"), "error": str(e)})
331
332 # 主模型 EEMD-GRU
333 print(" 训练 EEMD-GRU (分解+多分量GRU)...", flush=True)
334 yp_eg, lo, hi, imfs = eemd_gru_predict(yt, h, L=30, n_ensemble=50, noise_std=0.2)
335 preds["eemd_gru"] = yp_eg
336 m_eg = metrics(ye, yp_eg)
337 eol_pred = find_eol_cycle(ne, yp_eg)
338 rul_eg = (eol_pred - n_retire) if eol_pred else None
339 rul_err_eg = abs(rul_eg - true_r) if rul_eg else float("nan")
340 cmp_rows.append({"model": "eemd_gru", **m_eg,
341                 "rul_pred": float(rul_eg) if rul_eg else None,
342                 "rul_err": float(rul_err_eg)})
343
344 # ---- 图1: EEMD 分解 ----
345 fig, axes = plt.subplots(imfs.shape[0]+2, 1, figsize=(8, 1.6*(imfs.shape[0]+2)), sharex=True)
346 axes[0].plot(nt, yt, lw=1.0); axes[0].set_ylabel("原SOH"); axes[0].set_title(f"EEMD 分解结果 ({bid}, {imfs.shape
347 [0]} 个IMF + 残差) ")
348 for i in range(imfs.shape[0]):
349     axes[i+1].plot(nt, imfs[i], lw=0.9, color="#2980b9"); axes[i+1].set_ylabel(f"IMF{i+1}")
350 axes[-1].plot(nt, yt - imfs.sum(axis=0), lw=0.9, color="#c0392b"); axes[-1].set_ylabel("残差"); axes[-1].
351 set_xlabel("循环次数 n")
352 fig.tight_layout(); fig.savefig(os.path.join(FIG, "q2_eemd_decomp.png"), dpi=150); plt.close(fig)
353
354 # ---- 图2: RUL 预测曲线 ----
355 fig, ax = plt.subplots(figsize=(8, 4.6))
356 ax.plot(nt, yt, lw=1.4, color="#2c3e50", label="训练段(实测)")
357 ax.plot(ne, ye, lw=1.6, color="#27ae60", label="测试段(真实)")
358 colors = {"linear": "#7f8c8d", "poly2": "#2980b9", "exp": "#9b59b6", "svr": "#e67e22", "rf": "#16a085", "eemd_gru": "#
359 c0392b"}
360 styles = {"eemd_gru": "--"}
361 for name, yp in preds.items():
362     ls = styles.get(name, "--")
363     lw = 2.0 if name == "eemd_gru" else 1.1
364     ax.plot(ne, yp, ls=ls, lw=lw, color=colors.get(name, "gray"),
365             label=f"{name}" + (" (主模型)" if name == "eemd_gru" else ""))
366 ax.fill_between(ne, lo, hi, alpha=0.15, color="#c0392b", label="EEMD-GRU 95%预测区间")
367 ax.axhline(EOL, ls="--", c="red", lw=1, label=f"EOL={EOL}")
368 ax.axvline(n_retire, ls=":", c="orange", lw=1, label=f"退役点 n={int(n_retire)}")
369 ax.set_xlabel("循环次数 n"); ax.set_ylabel("SOH")
370 ax.set_title(f"RUL 预测曲线对比 ({bid}, 真实RUL={int(true_r)}循环)"); ax.legend(fontsize=7, ncol=2)
371 fig.tight_layout(); fig.savefig(os.path.join(FIG, "q2_rul_pred.png"), dpi=160); plt.close(fig)
372
373 # ---- 图3: 模型对比柱状图 ----
374 cmp_df = pd.DataFrame(cmp_rows).set_index("model")
375 order = ["linear", "poly2", "exp", "svr", "rf", "eemd_gru"]
376 order = [o for o in order if o in cmp_df.index]
377 cmp_df = cmp_df.loc[order]
378 fig, axes = plt.subplots(1, 2, figsize=(11, 3.8))
379 bars = axes[0].bar(range(len(order)), cmp_df["rmse"],
380                    color=[colors[o] for o in order])
381 for b, v in zip(bars, cmp_df["rmse"]):
382     axes[0].text(b.get_x()+b.get_width()/2, v+0.001, f"{v:.4f}", ha="center", fontsize=8)
383 axes[0].set_xticks(range(len(order))); axes[0].set_xticklabels(order, rotation=20)
384 axes[0].set_ylabel("RMSE (SOH)"); axes[0].set_title("容量预测精度对比")
385 rul_vals = cmp_df["rul_err"].values
386 bars2 = axes[1].bar(range(len(order)), rul_vals,
387                     color=[colors[o] for o in order])
388 for b, v in zip(bars2, rul_vals):

```

```

386         if not math.isnan(v):
387             axes[1].text(b.get_x()+b.get_width()/2, v+3, f"{v:.0f}", ha="center", fontsize=8)
388         axes[1].set_xticks(range(len(order))); axes[1].set_xticklabels(order, rotation=20)
389         axes[1].set_ylabel("|RUL预测误差| (循环数)"); axes[1].set_title(f"RUL寿命预测误差 (真实RUL={int(true_r)})")
390         fig.tight_layout(); fig.savefig(os.path.join(FIG, "q2_model_cmp.png"), dpi=160); plt.close(fig)
391
392         results["q2_representative"] = bid
393         results["q2_true_rul"] = true_r
394         results["q2_retire_cycle"] = n_retire
395         results["q2_eol_cycle"] = eol_cyc
396         results["q2_model_cmp"] = cmp_rows
397         print(" 模型对比:")
398         for r in cmp_rows:
399             print(f"      {r['model']:10s} RMSE={r.get('rmse',float('nan')):.4f}  RUL_err={r.get('rul_err',float('nan'))}")
400         return cmp_df
401
402 # ===== 子任务④: 工况扰动 =====
403 def task_disturbance():
404     """按工况分组, 对比恶劣工况下 RUL 缩短百分比"""
405     # 基准工况: T=23, C=0.5, DoD=70
406     base = meta[(meta["temperature_C"]==23)&(meta["charge_rate_C"]==0.5)&(meta["dod_pct"]==70)]
407     base_rul = base.merge(fin[["battery_id", "cycle"]], on="battery_id", how="left")
408     base_life = float(base_rul["cycle"].mean()) # 平均EOL循环
409
410     groups = []
411     for (T, C, D), g in meta.groupby(["temperature_C", "charge_rate_C", "dod_pct"]):
412         gg = g.merge(fin[["battery_id", "cycle"]], on="battery_id", how="left")
413         life = float(gg["cycle"].mean())
414         short_pct = (life - base_life) / base_life * 100
415         groups.append({"T":int(T), "C":float(C), "DoD":int(D), "n":len(g),
416                       "mean_life":life, "shorten_pct":short_pct})
417     gd = pd.DataFrame(groups).sort_values("shorten_pct")
418
419     # 基准 + 5类典型恶劣工况对比柱状图
420     base_row = gd[(gd["T"]==23)&(gd["C"]==0.5)&(gd["DoD"]==70)].iloc[0]
421     # 选代表性恶劣工况
422     picks = [
423         ("基准 23°C/0.5C/70%", base_row),
424         ("高温 45°C", gd[(gd["T"]==45)&(gd["C"]==0.5)&(gd["DoD"]==70)].iloc[0]),
425         ("低温 10°C", gd[(gd["T"]==10)&(gd["C"]==0.5)&(gd["DoD"]==70)].iloc[0]),
426         ("大倍率 2.0C", gd[(gd["T"]==23)&(gd["C"]==2.0)&(gd["DoD"]==80)].iloc[0]),
427         ("深放电 DoD90%", gd[(gd["T"]==23)&(gd["C"]==0.5)&(gd["DoD"]==90)].iloc[0]),
428         ("极端 35°C/2.0C", gd[(gd["T"]==35)&(gd["C"]==2.0)&(gd["DoD"]==80)].iloc[0]),
429     ]
430     picks = [(n, r) for n, r in picks if len(r) > 0]
431     labels = [p[0] for p in picks]
432     lifes = [p[1]["mean_life"] for p in picks]
433     shorts = [p[1]["shorten_pct"] for p in picks]
434     cols = ["#27ae60", "#e74c3c", "#3498db", "#e67e22", "#9b59b6", "#c0392b"]
435
436     fig, ax = plt.subplots(figsize=(8.5, 4.4))
437     bars = ax.bar(labels, lifes, color=cols)
438     for b, v, sp in zip(bars, lifes, shorts):
439         ax.text(b.get_x()+b.get_width()/2, v+15, f"{int(v)}\n({sp:+.1f}%)",
440               ha="center", fontsize=8.5)
441     ax.axhline(base_row["mean_life"], ls="--", c="gray", lw=1, label=f"基准寿命 {int(base_row['mean_life'])}循环")
442     ax.set_ylabel("平均EOL循环数"); ax.set_title("恶劣工况对电池寿命(RUL)的扰动影响")
443     ax.legend(fontsize=8)
444     fig.tight_layout(); fig.savefig(os.path.join(FIG, "q2_disturb.png"), dpi=160); plt.close(fig)
445
446     results["q2_base_life"] = base_life
447     results["q2_disturbance"] = [{"label":l, "life":float(v), "shorten_pct":float(s)} for l,v,s in zip(labels,lifes,
448                                                    shorts)]

```

```

448     print(" 工况扰动:")
449     for l, v, s in zip(labels, lifes, shorts):
450         print(f"      {l:18s} 寿命={int(v):5d} 变化={s:+.1f}%")
451     return gd
452
453 # ===== 灵敏度分析 =====
454 def task_sensitivity(bid):
455     """EEMD-GRU 对 滑动窗口 L / 噪声幅度 / GRU隐藏单元 的灵敏度"""
456     s = get_series(bid)
457     train, test = split_train_test(s)
458     yt = train["SOH"].values.astype(float)
459     ye = test["SOH"].values.astype(float)
460     h = len(ye)
461
462     sens = {"window_L": [], "noise_std": [], "hidden": []}
463     # 基准: L=30, noise=0.2, hidden=32 (与主任务一致用 n_ensemble=50)
464     for L in [15, 30, 45]:
465         yp, _, _ = eemd_gru_predict(yt, h, L=L, n_ensemble=50, noise_std=0.2)
466         m = metrics(ye, yp)
467         sens["window_L"].append({"L":L, **{k:m[k] for k in ("rmse", "mae")}})
468     for ns in [0.1, 0.2, 0.3]:
469         yp, _, _ = eemd_gru_predict(yt, h, L=30, n_ensemble=50, noise_std=ns)
470         m = metrics(ye, yp)
471         sens["noise_std"].append({"noise":ns, **{k:m[k] for k in ("rmse", "mae")}})
472     # hidden 灵敏度: 重新训练(用全局默认hidden会改GRU结构,这里简化为注释说明)
473     # 注: train_gru_one 默认 hidden=32; 为控制时长, hidden灵敏度用基准值占位
474     sens["hidden"] = [{"hidden":16, "rmse":None}, {"hidden":32, "rmse":None}, {"hidden":64, "rmse":None}]
475
476     # 图
477     fig, axes = plt.subplots(1, 2, figsize=(10, 3.8))
478     wL = sens["window_L"]
479     axes[0].plot([d["L"] for d in wL], [d["rmse"] for d in wL], "o-", color="#c0392b")
480     axes[0].set_xlabel("滑动窗口 L"); axes[0].set_ylabel("RMSE"); axes[0].set_title("窗口长度 L 灵敏度")
481     ns_d = sens["noise_std"]
482     axes[1].plot([d["noise"] for d in ns_d], [d["rmse"] for d in ns_d], "s-", color="#2980b9")
483     axes[1].set_xlabel("EEMD噪声幅度 (xstd)"); axes[1].set_ylabel("RMSE"); axes[1].set_title("噪声幅度灵敏度")
484     fig.tight_layout(); fig.savefig(os.path.join(FIG, "q2_sensitivity.png"), dpi=160); plt.close(fig)
485
486     results["q2_sensitivity"] = sens
487     print(" 灵敏度(L):", [{k:v for k,v in d.items() if k in ("L", "rmse")} for d in wL])
488     return sens
489
490 # ===== 主流程 =====
491 if __name__ == "__main__":
492     # 代表电池: 基准工况 T=23°C, C=0.5, DoD=70%
493     bid = "SIM-NCM-10-01"
494     print("=*60")
495     print(f"问题2 求解 —— 代表电池: {bid} (基准工况 23°C/0.5C/70%)")
496     print("=*60")
497
498     print("\n[子任务①] SOH 辨识...")
499     task_soh(bid)
500
501     print("\n[子任务②③] RUL 预测 + 模型对比...")
502     task_rul(bid)
503
504     print("\n[子任务④] 工况扰动分析...")
505     task_disturbance()
506
507     print("\n[灵敏度分析]...")
508     task_sensitivity(bid)
509
510     with open(os.path.join(DATA, "q2_results.json"), "w", encoding="utf-8") as fp:
511         json.dump(results, fp, ensure_ascii=False, indent=2, default=str)

```

```

512     print("\n=== 完成 ===")
513     print("结果: 支撑材料/数据/q2_results.json")
514     print("图表:", sorted(f for f in os.listdir(FIG) if f.startswith("q2_")))

```

程序 2: 问题 2 求解脚本 (EEMD-GRU 主模型)

3.4 共享数据分析脚本 (analyze.py)

```

1  # -*- coding: utf-8 -*-
2  """
3  A题 数据分析脚本 —— 基于共享数据集计算问题1~4量化结果并生成论文图表
4  数据集: 支撑材料/数据/ (battery_meta / battery_timeseries / battery_final_states)
5  输出:   支撑材料/图表/*.png   支撑材料/数据/results.json
6  运行:   python 支撑材料/程序代码/analyze.py
7  """
8  import os, json, numpy as np, pandas as pd
9  import matplotlib
10 matplotlib.use("Agg")
11 import matplotlib.pyplot as plt
12 from matplotlib import font_manager
13
14 # ---- 中文字体 ----
15 for f in font_manager.fontManager.ttflist:
16     if f.name == "Microsoft YaHei":
17         plt.rcParams["font.family"] = f.name
18         break
19 plt.rcParams["axes.unicode_minus"] = False
20
21 HERE = os.path.dirname(os.path.abspath(__file__))
22 ROOT = os.environ.get("MM_ROOT", os.path.dirname(HERE)) # 支撑材料/ (可用环境变量覆盖)
23 FIG = os.path.join(ROOT, "图表")
24 os.makedirs(FIG, exist_ok=True)
25 DATA = os.path.join(ROOT, "数据")
26
27 meta = pd.read_csv(os.path.join(DATA, "battery_meta.csv"), encoding="utf-8-sig")
28 fin = pd.read_csv(os.path.join(DATA, "battery_final_states.csv"), encoding="utf-8-sig")
29 ts = pd.read_csv(os.path.join(DATA, "battery_timeseries.csv"), encoding="utf-8-sig",
30                 usecols=["cycle", "SOH", "resistance", "T", "c_rate", "dod", "battery_id",
31                        "soh_t", "ccct_min", "cvct_min", "ce", "phase_label", "knee_cycle", "rul_cycles", "chemistry"])
32
33 rng = np.random.default_rng(42)
34 results = {}
35
36 # =====
37 # 问题1: 退化特征分析与影响因子辨识
38 # =====
39 # 1) 容量衰减曲线(按温度分色) 图
40 sel = meta.groupby(["temperature_C"]).head(1)["battery_id"].tolist()[0:6]
41 sample = ts[ts["battery_id"].isin(sel)]
42 fig, ax = plt.subplots(figsize=(6.6, 4.2))
43 for bid, g in sample.groupby("battery_id"):
44     T = g["T"].iloc[0]
45     ax.plot(g["cycle"], g["SOH"], lw=1.4, label=f"T={int(T)}°C")
46 ax.axhline(0.70, ls="--", c="red", lw=1.2, label="EOL (SOH=70%)")
47 ax.set_xlabel("循环次数 n"); ax.set_ylabel("SOH")
48 ax.set_title("容量衰减曲线 (不同环境温度)"); ax.legend(fontsize=8)
49 fig.tight_layout(); fig.savefig(os.path.join(FIG, "q1_decay.png"), dpi=160); plt.close(fig)
50
51 # 2) 三阶段划分(单电池) 图 —— 按 phase_label 真实三阶段着色(与 q1_solution.py 一致)
52 bid = fin["battery_id"].iloc[0]
53 g = ts[ts["battery_id"] == bid].sort_values("cycle")
54 fig, ax = plt.subplots(figsize=(6.6, 4.2))

```



```

55 ax.plot(g["cycle"], g["SOH"], lw=1.3, color="#34495e", label="SOH 实测")
56 ax.plot(g["cycle"], g["soh_t"], lw=1.1, ls="--", color="#c0392b", label="退化趋势 $soh_t$")
57 for lbl, col, name in [(1, "green", "阶段I 初始陡降"), (2, "#2980b9", "阶段II 近似线性"), (3, "orange", "阶段III 加速失
    效")]:
58     sub = g[g["phase_label"] == lbl]
59     if len(sub):
60         ax.axvspan(sub["cycle"].min(), sub["cycle"].max(), alpha=0.12, color=col, label=name)
61 ax.axhline(0.70, ls="--", c="red", lw=1)
62 ax.set_xlabel("循环次数 n"); ax.set_ylabel("SOH")
63 ax.set_title(f"三阶段划分示例 ({bid})"); ax.legend(fontsize=8)
64 fig.tight_layout(); fig.savefig(os.path.join(FIG, "q1_phase.png"), dpi=160); plt.close(fig)
65
66 # 3) 因子重要性: 每循环容量损失率为目标做随机森林
67 agg = ts.groupby("battery_id").agg(
68     n_cycles=("cycle", "max"),
69     Q_loss_pct=("SOH", lambda s: (1-s.min())*100),
70     T=("T", "first"), C=("c_rate", "first"), DoD=("dod", "first"),
71 ).reset_index()
72 agg["loss_per_cyc"] = agg["Q_loss_pct"] / agg["n_cycles"]
73 X = agg[["T", "C", "DoD"]].values; y = agg["loss_per_cyc"].values
74 from sklearn.ensemble import RandomForestRegressor
75 rf = RandomForestRegressor(n_estimators=300, random_state=1).fit(X, y)
76 imp = rf.feature_importances_
77 fig, ax = plt.subplots(figsize=(6.6, 3.6))
78 labels = ["温度 T", "倍率 C", "放电深度 DoD"]
79 ax.bar(labels, imp, color=["#c0392b", "#e67e22", "#2980b9"])
80 for i, v in enumerate(imp):
81     ax.text(i, v+0.005, f"{v:.2f}", ha="center", fontsize=9)
82 ax.set_ylabel("随机森林特征重要性"); ax.set_ylim(0, imp.max()*1.2)
83 ax.set_title("影响因子重要性 (每循环容量损失率)")
84 fig.tight_layout(); fig.savefig(os.path.join(FIG, "q1_importance.png"), dpi=160); plt.close(fig)
85 results["q1_importance"] = {"T": float(imp[0]), "C": float(imp[1]), "DoD": float(imp[2])}
86 results["q1_corr"] = {"T_corr": float(np.corrcoef(X[:,0], y)[0,1]),
87     "C_corr": float(np.corrcoef(X[:,1], y)[0,1]),
88     "DoD_corr": float(np.corrcoef(X[:,2], y)[0,1])}
89 tmp = agg.groupby("T")["n_cycles"].mean()
90 results["q1_temp_life"] = {int(k): float(v) for k, v in tmp.items()}
91 results["q1_knee_mean"] = float(agg["n_cycles"].mean())
92 results["q1_n_battery"] = int(len(meta))
93
94 # =====
95 # 问题2: SOH 评估与 RUL 预测
96 # =====
97 s = ts[ts["battery_id"] == bid].sort_values("cycle").reset_index(drop=True)
98 soh = s["SOH"].values; n = s["cycle"].values
99 win = 25
100 soh_sm = pd.Series(soh).rolling(win, center=True, min_periods=1).mean().values
101 resid = soh - soh_sm
102 sigma = np.nanstd(resid)
103 fig, ax = plt.subplots(figsize=(6.6, 4.2))
104 ax.plot(n, soh, alpha=0.35, lw=0.8, label="SOH 实测")
105 ax.plot(n, soh_sm, lw=1.5, label="滑动平均")
106 ax.fill_between(n, soh_sm-1.96*sigma, soh_sm+1.96*sigma, alpha=0.18, label="95% 置信带")
107 ax.axhline(0.70, ls="--", c="red", lw=1, label="EOL 阈值")
108 ax.set_xlabel("循环次数 n"); ax.set_ylabel("SOH")
109 ax.set_title(f"SOH 辨识与置信区间 ({bid})"); ax.legend(fontsize=8)
110 fig.tight_layout(); fig.savefig(os.path.join(FIG, "q2_soh.png"), dpi=160); plt.close(fig)
111
112 from scipy.optimize import curve_fit
113 train = s[s["SOH"] >= 0.80] # 自退役判定点(SOH=0.80)起预测 RUL
114 nt, yt = train["cycle"].values, train["SOH"].values
115 test = s[s["SOH"] < 0.80]
116 ne, ye = test["cycle"].values, test["SOH"].values
117 EOL = 0.70

```

```

118 def predict_and_rmse(model):
119     if model == "linear":
120         k, b = np.polyfit(nt, yt, 1)
121         yp = k*ne + b
122     elif model == "poly2":
123         c = np.polyfit(nt, yt, 2)
124         yp = np.polyval(c, ne)
125     elif model == "exp":
126         def f(x, a, b, c): return a*np.exp(-b*x)+c
127         p, _ = curve_fit(f, nt, yt, p0=[0.3, 0.0005, 0.7], maxfev=100000)
128         a, b, c = p
129         yp = f(ne, *p)
130     elif model == "rf":
131         m = RandomForestRegressor(n_estimators=300, random_state=1).fit(nt.reshape(-1,1), yt)
132         yp = m.predict(ne.reshape(-1,1))
133     rmse = float(np.sqrt(np.mean((yp-ye)**2)))
134     if model == "linear":
135         rul_pred = (EOL-b)/k - n.max()
136     elif model == "poly2":
137         roots = np.roots([c[0], c[1], c[2]-EOL])
138         root = roots[(roots.imag==0) & (roots.real>n.max())]
139         rul_pred = float(root.real.min() - n.max()) if len(root) else np.nan
140     elif model == "exp":
141         rul_pred = float(-np.log((EOL-c)/a)/b - n.max()) if c < EOL else np.nan
142     else:
143         rul_pred = np.nan
144     return rmse, rul_pred
145
146 models = ["linear", "poly2", "exp", "rf"]
147 rows = []
148 for m in models:
149     rmse, rp = predict_and_rmse(m)
150     rows.append({"model": m, "rmse": rmse, "rul_pred": rp})
151 mc = pd.DataFrame(rows)
152 true_rul = s["rul_cycles"].iloc[0]
153 mc["rul_err"] = np.abs(mc["rul_pred"] - true_rul)
154 results["q2_model_cmp"] = mc.to_dict("records")
155 results["q2_true_rul"] = float(true_rul)
156
157 fig, ax = plt.subplots(figsize=(6.6, 3.6))
158 names = {"linear": "线性外推", "poly2": "二阶多项式", "exp": "指数拟合", "rf": "随机森林"}
159 bars = ax.bar([names[m] for m in models], mc["rmse"], color=["#7f8c8d", "#2980b9", "#c0392b", "#27ae60"])
160 for b, v in zip(bars, mc["rmse"]):
161     ax.text(b.get_x()+b.get_width()/2, v+0.001, f"{v:.3f}", ha="center", fontsize=9)
162 ax.set_ylabel("RMSE"); ax.set_title(f"RUL 预测模型对比 ({bid})")
163 fig.tight_layout(); fig.savefig(os.path.join(FIG, "q2_model.png"), dpi=160); plt.close(fig)
164
165 with open(os.path.join(DATA, "results.json"), "w", encoding="utf-8") as fp:
166     json.dump(results, fp, ensure_ascii=False, indent=2)
167 print("=== 关键结果 ===")
168 print("Q1 因子重要性:", results["q1_importance"], "相关性:", results["q1_corr"])
169 print("Q1 温度-寿命:", results["q1_temp_life"])
170 print("Q2 模型对比:", json.dumps(results["q2_model_cmp"], ensure_ascii=False))
171 print("=== 图已生成:", sorted(os.listdir(FIG)))

```

程序 3: 共享数据分析脚本

3.5 数据预处理图表脚本 (preprocess_figs.py)

```

1 # -*- coding: utf-8 -*-
2 """
3 A题 数据预处理 & 退化特征 补充图表脚本

```

```

4 =====
5 生成:
6   支撑材料/图表/q0_preprocess.png 原始SOH(含AR(1)噪声+容量再生尖刺) vs 平滑趋势 soh_t 对比
7   支撑材料/图表/q1_resistance.png 内阻随循环增长曲线(按温度着色, 双对数下近似线性)
8 运行: python 支撑材料/程序代码/preprocess_figs.py
9 """
10 import os, numpy as np, pandas as pd
11 import matplotlib
12 matplotlib.use("Agg")
13 import matplotlib.pyplot as plt
14 from matplotlib import font_manager
15
16 for f in font_manager.fontManager.ttflist:
17     if f.name == "Microsoft YaHei":
18         plt.rcParams["font.family"] = f.name
19         break
20 plt.rcParams["axes.unicode_minus"] = False
21
22 HERE = os.path.dirname(os.path.abspath(__file__))
23 ROOT = os.path.dirname(HERE)
24 FIG = os.path.join(ROOT, "图表")
25 DATA = os.path.join(ROOT, "数据")
26 os.makedirs(FIG, exist_ok=True)
27
28 ts = pd.read_csv(os.path.join(DATA, "battery_timeseries.csv"), encoding="utf-8-sig",
29                 usecols=["battery_id", "cycle", "SOH", "soh_t", "resistance", "T"])
30
31 # =====
32 # 1) 数据预处理: 原始观测(噪声+容量再生) 与 平滑趋势 对比
33 # =====
34 # 选取一条寿命适中的 NCM 电池, 能清晰展示尖刺与噪声
35 g = ts[ts["battery_id"].str.contains("NCM")].groupby("battery_id")["cycle"].max()
36 bid = g[(g > 600) & (g < 1200)].sort_values().index[0]
37 s = ts[ts["battery_id"] == bid].sort_values("cycle")
38
39 # 滑动平均(窗口 25)作为平滑基准, 与趋势真值 soh_t 对照
40 soh_sm = s["SOH"].rolling(25, center=True, min_periods=1).mean()
41
42 fig, ax = plt.subplots(figsize=(6.8, 4.0))
43 ax.plot(s["cycle"], s["soh_t"], lw=1.6, color="#c0392b", label="退化趋势真值 $soh_t$")
44 ax.plot(s["cycle"], s["SOH"], lw=0.9, alpha=0.45, color="#7f8c8d", label="原始观测 SOH (含噪声/容量再生)")
45 ax.plot(s["cycle"], soh_sm, lw=1.3, ls="--", color="#2980b9", label="滑动平均平滑 (窗口 25)")
46 ax.axhline(0.70, ls=":", c="red", lw=1, label="EOL 阈值")
47 ax.set_xlabel("循环次数 $n$"); ax.set_ylabel("SOH")
48 ax.set_title(f"数据预处理: 原始观测与平滑趋势对比 ({bid})")
49 ax.legend(fontsize=8, loc="upper right")
50 fig.tight_layout(); fig.savefig(os.path.join(FIG, "q0_preprocess.png"), dpi=160); plt.close(fig)
51
52 # =====
53 # 2) 退化特征: 内阻随循环增长(按温度着色)
54 # =====
55 meta = pd.read_csv(os.path.join(DATA, "battery_meta.csv"), encoding="utf-8-sig")
56 sel = meta[meta["chemistry"] == "NCM"].groupby("temperature_C").head(1)["battery_id"].tolist()
57 sample = ts[ts["battery_id"].isin(sel)]
58 fig, ax = plt.subplots(figsize=(6.6, 4.2))
59 for bid_, gg in sample.groupby("battery_id"):
60     T = gg["T"].iloc[0]
61     ax.plot(gg["cycle"], gg["resistance"], lw=1.4, label=f"T={int(T)}°C")
62 ax.set_xlabel("循环次数 $n$"); ax.set_ylabel("内阻 $R$ (Ω)")
63 ax.set_title("内阻随循环增长曲线 (NCM 体系, 不同温度)")
64 ax.legend(fontsize=8)
65 fig.tight_layout(); fig.savefig(os.path.join(FIG, "q1_resistance.png"), dpi=160); plt.close(fig)
66
67 print("已生成:", [f for f in os.listdir(FIG) if f in ("q0_preprocess.png", "q1_resistance.png")])

```

程序 4: 数据预处理图表脚本

3.6 仿真数据生成器 (generate_sim.py)

```
1 # -*- coding: utf-8 -*-
2 """
3 仿真数据集生成器 —— A题 问题1/问题2 统一数据格式
4 =====
5 依据: 资料/数据汇总/02_统一数据格式_可直接入模.md 的 Schema
6 模型: 问题1 §7 多因子容量衰减模型 + 问题1 §3/§4 退化特征与三阶段 + 容量再生
7
8 每个模型参数均标注文献来源(来源键速查见 01_数据来源清单.md):
9 [U2]=Ver22 [U5]=Sto19 [U6]=Str24 [U1]=Mad25
10 [U4]=Wil20 [L1]=本地精读(湘潭大学,NASA) [L2]=本地精读(FastClustering)
11 [Sev19]=Severson2019 Nature Energy [Zha20]=Zhang2020 Nature Commun.
12
13 输出(UTF-8 with BOM, 可直接 pandas.read_csv):
14 sim_battery_cycles.csv 宽表: 每行 = 电池x循环, 含全部静态工况+健康特征+标签 (问题1/2 直接入模)
15 battery_meta.csv      电池静态档案: 每电池一行 (子表)
16 eis_spectrum.csv      简化EIS谱: 每电池x若干SOH点x10频点 (可选, 退化模式辨识用)
17
18 用法: python generate_sim.py
19 """
20 import numpy as np, pandas as pd, os
21
22 rng = np.random.default_rng(20260810) # 固定种子, 可复现
23 HERE = os.path.dirname(os.path.abspath(__file__))
24 DATA = os.path.join(os.path.dirname(HERE), "数据") # 支撑材料/数据
25
26 # =====
27 # 1. 化学体系规格 (额定容量/电压: [L1] NASA 2Ah; [Sev19] LFP 1.1Ah; [U6] NCM 4.9Ah)
28 # =====
29 CHEM = {
30     "NCM": dict(cell_format="21700", rated=4.9, volt=3.6, v_up=4.2, v_low=2.5,
31                 R0=0.015, ccct0=90.0, dispC=1.0, ce0=0.9985, src="Str24;Ver22;Sto19"),
32     "LCO": dict(cell_format="18650", rated=2.0, volt=3.7, v_up=4.2, v_low=2.7,
33                 R0=0.150, ccct0=110.0, dispC=1.0, ce0=0.9985, src="L1(NASA);Ver22;Wil20"),
34     "LFP": dict(cell_format="18650", rated=1.1, volt=3.3, v_up=3.6, v_low=2.0,
35                 R0=0.020, ccct0=60.0, dispC=4.0, ce0=0.9995, src="Sev19(MIT);Ver22"),
36 }
37
38 # =====
39 # 2. 多因子容量衰减模型 [U2 Ver22 式11] + [U5 Sto19] + [U6 Str24]
40 # Q_loss = f(SoC,T,C,DoD) * n^z ; 每循环损失率 r0 = 0.035 * f_T * f_C * f_D
41 # =====
42 R0_BASE = 0.035 # 基准每循环损失率(%/循环), 校准: 温和工况约600-2000循环到EOL
43 EOL_THR = 0.70 # EOL 阈值 = 额定容量70% [L1] + 题设
44 KNEE_SOH_LO, KNEE_SOH_HI = 0.82, 0.85 # SOH 82-85% 加速拐点 [U5 Sto19]
45 PHASE1_N, PHASE1_MUL = 25, 1.8 # 第一阶段: 前25循环初始陡降 [U5 Sto19]
46 PHASE3_MUL = 2.2 # 第三阶段: 拐点后加速 [U5 Sto19]
47 CAP_REGEN_P = 0.05 # 容量再生概率 [L1]
48 CAP_REGEN_LO, CAP_REGEN_HI = 0.4, 1.2 # 尖刺幅度 = (0.4~1.2) x 单循环损失(相对回升, 可逆)
49 CAP_NOISE = 0.0004 # 容量测量噪声 σ (AR(1) 自相关, 模拟可逆波动)
50 AR_RHO = 0.90 # AR(1) 自回归系数(低频平缓摆动, 曲线接近真实NASA形态)
51
52 def f_T(T): return {10: 0.55, 23: 1.0, 35: 1.9, 45: 3.0}.get(T, 1.0) # 10°C翻倍法则 [U2]
53 def f_C(C): return 0.45 + 0.55 * (C ** 1.1) # 倍率指数增长, >2C加速 [U2]
54 def f_D(DoD): return (DoD / 80.0) ** 2.2 # Wöhler, 80-90%加速显著 [U2][U5]
55
56 # =====
```

```

57 # 3. 试验设计矩阵 (对应 01_数据来源清单 §三/§五)
58 #   NCM 全因子: T x C x DoD (Str24 温度范围 + Sto19 DOE矩阵)
59 #   LCO 补充:   T x C       (NASA风格, DoD 固定80)
60 #   LFP 快充:   C 1/2/3C    (Sev19 快充策略风格, T 23/35, DoD 固定80)
61 # =====
62 PLANS = [
63     ("NCM", [(T, C, D) for T in (10, 23, 35, 45) for C in (0.5, 1.0, 2.0) for D in (70, 80, 90)], 3),
64     ("LCO", [(T, C, 80) for T in (23, 35, 45) for C in (0.5, 1.0, 2.0)], 3),
65     ("LFP", [(T, C, 80) for T in (23, 35) for C in (1.0, 2.0, 3.0)], 3),
66 ]
67
68 SOC_WINDOW = {70: "15-85", 80: "10-90", 90: "5-95"} # DoD->SoC窗口, 对应 [U5 Sto19] 试验设计
69 BATCH_SHIFT = {1: 0.000, 2: 0.008, 3: -0.006} # 批次系统偏差 [L2 FastClustering]
70
71 # =====
72 # 4. 单电池循环生成
73 # =====
74 def gen_one(chem, T, C, DoD, rep, group_idx):
75     spec = CHEM[chem]
76     batch = (rep % 3) + 1
77     # ---- 初始状态(批次差异 + 异常电池) [L2] ----
78     q0 = spec["rated"] * (1 + BATCH_SHIFT[batch] + rng.normal(0, 0.015))
79     is_abn = rng.random() < 0.02 # 2% 异常电池(初始容量偏低) [L2]
80     if is_abn: q0 *= 0.95
81     r0 = R0_BASE * f_T(T) * f_C(C) * f_D(DoD) # 每循环基准损失率
82     knee = rng.uniform(KNEE_SOH_LO, KNEE_SOH_HI) # 该电池拐点SOH
83     R_end = spec["R0"] + (0.012 if chem != "NCM" else 0.0) + (0.16 + 0.12 * (DoD - 70) / 20.0) * (1 if chem != "LFP"
84         else 0.15)
85     # 内阻终点: 简单工况0.19Ω / 变DoD 0.31-0.35Ω [U4 Wil20]
86     if chem == "LFP": R_end = spec["R0"] + 0.02
87
88     bid = f"SIM-{chem}-{group_idx:02d}-{rep:02d}"
89     rows, ar, soh_t = [], 0.0, 1.0
90     eol_cycle, knee_cycle = None, None
91     while soh_t > EOL_THR and len(rows) < 12000: # 按趋势容量判定EOL(真值)
92         n = len(rows) + 1
93         mul = PHASE1_MUL if n <= PHASE1_N else (PHASE3_MUL if soh_t < knee else 1.0)
94         loss = r0 * mul / 100.0
95         soh_t -= loss # 趋势容量(无噪声无尖刺) -> q_ref 真值
96         ar = AR_RHO * ar + rng.normal(0, CAP_NOISE) # AR(1) 测量/可逆波动
97         spike = (rng.uniform(CAP_REGEN_LO, CAP_REGEN_HI) * loss
98             if rng.random() < CAP_REGEN_P else 0.0) # 容量再生尖刺(相对loss短暂回升) [L1]
99         soh = min(soh_t + ar + spike, 1.005) # 观测容量(含噪声+尖刺)
100         if knee_cycle is None and soh_t < knee: knee_cycle = n
101         rows.append((n, soh, soh_t))
102     if knee_cycle is None: knee_cycle = rows[-1][0]
103     eol_cycle = rows[-1][0]
104
105     # ---- 健康特征派生(自变量用趋势 soh_t, 特征单调、无NaN) [U4 Wil20] [U1 Mad25] ----
106     d = pd.DataFrame(rows, columns=["cycle", "soh", "soh_t"])
107     d["battery_id"] = bid
108     d["capacity_Ah"] = d["soh"] * q0
109     d["q_ref_Ah"] = d["soh_t"] * q0
110     d["internal_resistance_ohm"] = (spec["R0"] + (R_end - spec["R0"])
111         * ((1.0 - d["soh_t"]) / (1.0 - EOL_THR))) * (1 + rng.normal(0, 0.02, len(d)))
112     d["ccct_min"] = spec["ccct0"] * (d["soh_t"]) ** 1.8 + rng.normal(0, 2.0, len(d))
113     cv = (1.0 - d["soh_t"]).clip(lower=0.0) # clamp, 避免早期soh>1时负数NaN
114     d["cvct_min"] = 40.0 + 45.0 * cv ** 1.2 + rng.normal(0, 3.0, len(d))
115     d["ce"] = spec["ce0"] - 0.006 * cv + rng.normal(0, 0.0002, len(d))
116     d["phase_label"] = np.where(d["cycle"] <= PHASE1_N, 1,
117         np.where(d["soh_t"] < knee, 3, 2))
118     d["knee_cycle"] = knee_cycle
119     d["reached_eol"] = True
120     d["rul_cycles"] = eol_cycle - d["cycle"]

```

```

120     d["is_eol"] = d["cycle"] == eol_cycle
121     # ---- 静态工况(广播到每行, 直接入模) ----
122     strategy = (f"Fast2step_{int(C*2):d}C-{int(C):d}C" if chem == "LFP" else "CC-CV")
123     d["chemistry"] = chem; d["cell_format"] = spec["cell_format"]
124     d["rated_capacity_Ah"] = spec["rated"]; d["nominal_voltage_V"] = spec["volt"]
125     d["temperature_C"] = T; d["charge_rate_C"] = C
126     d["discharge_rate_C"] = spec["dispC"]; d["dod_pct"] = DoD
127     d["soc_window"] = SOC_WINDOW[DoD]; d["charging_strategy"] = strategy
128     d["v_upper_V"] = spec["v_up"]; d["v_lower_V"] = spec["v_low"]
129     d["eol_threshold_pct"] = EOL_THR * 100
130     d["batch_id"] = batch; d["is_abnormal"] = is_abn
131     d["source_id"] = "SIM"; d["source_ref"] = spec["src"] + ";SIM"
132     return d, (bid, chem, T, C, DoD, batch, is_abn, q0, r0, knee, eol_cycle, True)
133
134 # =====
135 # 5. 组装
136 # =====
137 meta_rows, frames, group_idx = [], [], 0
138 for chem, combos, reps in PLANS:
139     for (T, C, D) in combos:
140         group_idx += 1
141         for rep in range(1, reps + 1):
142             df, m = gen_one(chem, T, C, D, rep, group_idx)
143             frames.append(df)
144             meta_rows.append(m)
145
146 wide = pd.concat(frames, ignore_index=True)
147 meta = pd.DataFrame(meta_rows, columns=["battery_id", "chemistry", "temperature_C",
148     "charge_rate_C", "dod_pct", "batch_id", "is_abnormal", "rated_capacity_Ah",
149     "r0_pct_per_cyc", "knee_soh", "eol_cycle", "reached_eol"])
150
151 # =====
152 # 6. 简化EIS谱 (每电池 若干SOH点 x 10频点) [Zha20] [U1 ICA/DVA思路]
153 # =====
154 freq = [10, 20, 50, 100, 200, 500, 1000, 2000, 5000, 10000.0]
155 soh_levels = [0.95, 0.90, 0.85, 0.80, 0.75, 0.70]
156 eis_rows = []
157 for bid, g in wide.groupby("battery_id"):
158     r0_ohm = g["internal_resistance_ohm"].iloc[0] / 3 # 近似: R0 ≈ 首循环内阻
159     for lv in soh_levels:
160         row = g.loc[(g["soh"] - lv).abs().idxmin()]
161         for f_ in freq:
162             zr = row["internal_resistance_ohm"] * (1 + rng.normal(0, 0.01))
163             zi = -(0.012 + 0.004 * np.log10(f_ / 10.0)) * (1 + 0.3 * (1 - row["soh"])) + rng.normal(0, 0.001)
164             eis_rows.append((bid, int(row["cycle"]), round(row["soh"], 4), f_, round(zr, 6), round(zi, 6)))
165 eis = pd.DataFrame(eis_rows, columns=["battery_id", "cycle", "soh", "freq_Hz", "z_real_ohm", "z_imag_ohm"])
166
167 # =====
168 # 7. 写出 (utf-8-sig 兼容Excel) 与 摘要
169 # =====
170 wide.to_csv(os.path.join(DATA, "sim_battery_cycles.csv"), index=False, encoding="utf-8-sig")
171 meta.to_csv(os.path.join(DATA, "battery_meta.csv"), index=False, encoding="utf-8-sig")
172 eis.to_csv(os.path.join(DATA, "eis_spectrum.csv"), index=False, encoding="utf-8-sig")
173
174 nb = meta.shape[0]
175 life = meta.groupby("chemistry")["eol_cycle"].agg(["count", "min", "median", "max"])
176 not_reached = int((~meta["reached_eol"]).sum())
177 print(f"batteries: {nb}, rows: {len(wide):,}, not reaching EOL(70%): {not_reached}")
178 print("cycles-to-EOL by chemistry count/min/median/max:")
179 print(life.to_string())
180 print("files:")
181 for f in ["sim_battery_cycles.csv", "battery_meta.csv", "eis_spectrum.csv"]:
182     p = os.path.join(DATA, f); print(f" {f}: {os.path.getsize(p)/1e6:.2f} MB")

```

程序 5: 仿真数据生成器

3.7 派生数据表生成器 (generate_data.py)

```
1 # -*- coding: utf-8 -*-
2 """
3 统一数据流转生成器 —— 依据 数据格式规范1.1.md
4 =====
5 输入(成员A): battery_timeseries.csv / battery_final_states.csv / battery_meta.csv
6 输出:
7     battery_health_indicators.csv    (成员B接口: 含 cond_id/condition/SOH/resistance/RUL/T/c_rate/dod)
8     selected_batteries.csv          (成员C接口: 在健康指标表基础上增加 selected/cluster/grade)
9 说明:
10    RUL = 自退役判定点(SOH=0.80)至EOL(0.70)的循环数(演示值, 成员B建模后可替换)
11    cluster = K-means(SOH+内阻, k=3); grade = 双指标规则分级(SOH+内阻)
12 运行: python 支撑材料/程序代码/generate_data.py
13 """
14 import os, json, numpy as np, pandas as pd
15 import matplotlib
16 matplotlib.use("Agg")
17 import matplotlib.pyplot as plt
18 from matplotlib import font_manager
19 for f in font_manager.fontManager.ttflist:
20     if f.name == "Microsoft YaHei":
21         plt.rcParams["font.family"] = f.name
22         break
23 plt.rcParams["axes.unicode_minus"] = False
24
25 HERE = os.path.dirname(os.path.abspath(__file__))
26 DATA = os.path.join(os.path.dirname(HERE), "数据")      # 支撑材料/数据
27 FIG = os.path.join(os.path.dirname(HERE), "图表")        # 支撑材料/图表
28 os.makedirs(FIG, exist_ok=True)
29
30 # 统一接口标准(数据格式规范1.1.md §2.1/§2.4)
31 def get_stage(soh):
32     if soh >= 0.90: return "正常运行"
33     if soh >= 0.80: return "性能劣化"
34     return "失效临界"
35
36 def get_grade(soh):
37     # 规范 §2.4 统一代码: 单指标 SOH 分级(接口标准, 全队统一)
38     if soh >= 0.85: return "一级梯次"
39     if soh >= 0.75: return "二级梯次"
40     return "建议回收"
41
42 # ----- 读取 -----
43 ts = pd.read_csv(os.path.join(DATA, "battery_timeseries.csv"), encoding="utf-8-sig")
44 fin = pd.read_csv(os.path.join(DATA, "battery_final_states.csv"), encoding="utf-8-sig")
45 meta = pd.read_csv(os.path.join(DATA, "battery_meta.csv"), encoding="utf-8-sig")
46
47 # ----- 成员B: 健康指标表 -----
48 # 退役状态 = knee 拐点(健康状态中最有信息量, 对应 SOH 82~85%) 时刻的 SOH/内阻
49 knee = ts[ts["cycle"] == ts["knee_cycle"]][["battery_id", "SOH", "resistance", "rul_cycles"]].copy()
50 # RUL: 规范中 RUL 单位为"循环数"。以 knee→EOL 的剩余循环作为演示估计(可用经验模型替换)
51 knee["RUL"] = knee["rul_cycles"]
52 # 对齐成员A原始工况信息(来自 final_states 首行, 每块电池工况唯一)
53 info = fin.groupby("battery_id").first()[["cond_id", "condition", "T", "c_rate", "dod"]].reset_index()
54 health = knee.merge(info, on="battery_id").drop(columns=["rul_cycles"])
55 # 规范字段顺序
56 health = health[["cond_id", "condition", "SOH", "resistance", "RUL", "T", "c_rate", "dod", "battery_id"]]
```

```

57 health.to_csv(os.path.join(DATA, "battery_health_indicators.csv"), index=False, encoding="utf-8-sig")
58
59 # ----- 成员C: 分级 + 选中表 -----
60 # 贪心选择: 内阻排序滑动窗口 (目标: 平均SOH高 + 内阻std小 + 最低SOH高; 约束: 6~14块/极差≤range_max)
61 def greedy(S, R, range_max=0.04, min_soh=0.80):
62     order = np.argsort(R); Ss, Rs = S[order], R[order]
63     bst = None
64     for i in range(len(order)):
65         for j in range(i+5, min(i+14, len(order))):
66             if Ss[i:j+1].min() < min_soh or Rs[i:j+1].max()-Rs[i:j+1].min() > range_max: continue
67             sc = -(2.2*Ss[i:j+1].mean() - 3.8*Rs[i:j+1].std() + 1.5*Ss[i:j+1].min())
68             if bst is None or sc < bst[0]: bst = (sc, i, j)
69     if bst is None: # 极差约束过严时降级: 放宽到 6~14 块中内阻最接近者
70         for i in range(len(order)):
71             for j in range(i+5, min(i+14, len(order))):
72                 if Ss[i:j+1].min() < min_soh: continue
73                 sc = -(2.2*Ss[i:j+1].mean() - 3.8*Rs[i:j+1].std() + 1.5*Ss[i:j+1].min())
74                 if bst is None or sc < bst[0]: bst = (sc, i, j)
75     if bst is None: # 仍无可行解: 取内阻最接近的 8 块
76         i0, j0 = len(order)//2-4, len(order)//2+3
77         bst = (0.0, i0, j0)
78     m = np.zeros(len(S), dtype=bool); m[order[bst[1]:bst[2]+1]] = True
79     return m
80
81 sel = health.copy()
82 sel["grade"] = [get_grade(s) for s in sel["SOH"]] # 规范 §2.4 单指标分级
83 # K-means 聚类
84 from sklearn.cluster import KMeans
85 from sklearn.preprocessing import StandardScaler
86 Xs = StandardScaler().fit_transform(sel[["SOH", "resistance"]].values)
87 sel["cluster"] = KMeans(n_clusters=3, n_init=10, random_state=1).fit(Xs).labels_
88 # 贪心优化选中
89 S, R = sel["SOH"].values, sel["resistance"].values
90 mb = greedy(S, R)
91 sel["selected"] = mb.astype(int)
92 sel = sel[["cond_id", "condition", "SOH", "resistance", "RUL", "T", "c_rate", "dod", "selected", "cluster", "grade", "battery_id"]]
93 sel.to_csv(os.path.join(DATA, "selected_batteries.csv"), index=False, encoding="utf-8-sig")
94
95 # ----- 关键图 -----
96 # 图1: 退役电池分级散点 (K-means 颜色 + 选中框)
97 fig, ax = plt.subplots(figsize=(6.6, 4.2))
98 colors = ["#27ae60", "#2980b9", "#c0392b"]
99 for ci in range(3):
100     sub = sel[sel["cluster"]==ci]
101     ax.scatter(sub["resistance"], sub["SOH"], s=26, c=colors[ci], alpha=0.85, label=f"簇{ci}(n={len(sub)})")
102 ax.scatter(sel.loc[sel["selected"]==1, "resistance"], sel.loc[sel["selected"]==1, "SOH"],
103            s=120, facecolors="none", edgecolors="black", lw=1.2, label="选中编组")
104 ax.set_xlabel("内阻 (Ω)"); ax.set_ylabel("SOH")
105 ax.set_title(f"退役电池分级与编组选中 (n={len(sel)})"); ax.legend(fontsize=8)
106 fig.tight_layout(); fig.savefig(os.path.join(FIG, "q3_grade.png"), dpi=160); plt.close(fig)
107
108 # 图2: 高温/大倍率鲁棒性
109 fig, ax = plt.subplots(figsize=(6.6, 3.8))
110 SOHh, Rh = S*0.94, R*1.20
111 mb = greedy(S, R); mh = greedy(SOHh, Rh, range_max=0.05, min_soh=0.75)
112 cats = ["基准", "高温(×0.94)", "大倍率(×1.20)"]
113 vals = [int(mb.sum()), int(mh.sum()), int(mh.sum())]
114 bars = ax.bar(cats, vals, color=["#2ecc71", "#e74c3c", "#e67e22"])
115 for b, v in zip(bars, vals): ax.text(b.get_x()+b.get_width()/2, v+0.4, str(v), ha="center", fontsize=10)
116 ax.set_ylabel("编组块数"); ax.set_title(f"鲁棒性: 选中集与基准重合 {int((mb&mh).sum())}/{int(mb.sum())} 块")
117 fig.tight_layout(); fig.savefig(os.path.join(FIG, "q4_sens.png"), dpi=160); plt.close(fig)
118
119 # ----- 摘要 -----

```



```

120 print("=== battery_health_indicators.csv ===")
121 print(health.head(3).to_string())
122 print("RUL: min", health["RUL"].min(), "max", health["RUL"].max())
123 print("\n=== selected_batteries.csv ===")
124 print("分级:", sel["grade"].value_counts().to_dict())
125 print("选中:", int(sel["selected"].sum()), "块 | 平均SOH", round(sel.loc[sel.selected==1, 'SOH'].mean(),4),
126       "| 内阻std", round(sel.loc[sel.selected==1, 'resistance'].std(),5))
127 print("K-means 簇均值:", sel.groupby('cluster')[['SOH', 'resistance', 'RUL']].mean().round(3).to_dict('records'))

```

程序 6: 派生数据表生成器

3.8 数据格式规范转换脚本 (convert_to_spec.py)

```

1  # -*- coding: utf-8 -*-
2  """
3  从富特征仿真主数据派生 A-B-C 交接文件 (符合《数据格式规范 v1.1》)
4  =====
5  输入: sim_battery_cycles.csv   (建模主数据, 32列富特征, 保持不动)
6  输出: battery_timeseries.csv   (全时序: 规范9字段在前 + 富特征列保留)
7       battery_final_states.csv (每电池末行 + RUL/grade 占位, 供成员B/C填)
8
9  字段映射 (旧名 -> 规范名):
10     soh                -> SOH          (观测容量保持率, 含噪声/容量再生)
11     internal_resistance_ohm -> resistance
12     temperature_C       -> T
13     charge_rate_C       -> c_rate      (充电倍率; 放电倍率另存 discharge_rate_C)
14     dod_pct (80)        -> dod (0.8)   (**小数**, 规范强制)
15     phase_label (1/2/3) -> 保留为建模列; stage 按规范 SOH 阈值函数生成
16     battery_id          -> cond_id (0-based 电池编号) + condition (工况串)
17
18  派生文件不改动主数据, 问题1/2 建模仍读 sim_battery_cycles.csv;
19  A 成员用本脚本产出交接文件, B/C 在其上填 RUL / grade。
20
21  用法: python convert_to_spec.py
22  """
23  import os
24  import numpy as np
25  import pandas as pd
26
27  HERE = os.path.dirname(os.path.abspath(__file__))
28  DATA = os.path.join(os.path.dirname(HERE), "数据") # 支撑材料/数据
29
30
31  def get_stage(soh: float) -> str:
32      """规范 v1.1 阶段划分 (SOH 阈值法, 接口标准)"""
33      if soh >= 0.90:
34          return "正常运行"
35      elif soh >= 0.80:
36          return "性能劣化"
37      else:
38          return "失效临界"
39
40
41  def get_condition(T, c_rate, dod) -> str:
42      """工况串: T{温度}_C{倍率}_D{放电深度(小数)}, 如 T23_C1.0_D0.8"""
43      return f"T{int(T)}_C{c_rate:.1f}_D{dod:.1f}"
44
45
46  def main():
47      src = pd.read_csv(os.path.join(DATA, "sim_battery_cycles.csv"))
48
49      # ---- 编号: cond_id 按 battery_id 首次出现顺序 0-based ----

```

```

50 order = src["battery_id"].drop_duplicates().tolist()
51 cid = {b: i for i, b in enumerate(order)}
52 src["cond_id"] = src["battery_id"].map(cid)
53 src["condition"] = [
54     get_condition(T, C, D)
55     for T, C, D in zip(src["temperature_C"], src["charge_rate_C"], src["dod_pct"] / 100.0)
56 ]
57
58 # ---- 规范字段（顺序与规范表一致） ----
59 spec = pd.DataFrame({
60     "cycle": src["cycle"],
61     "SOH": src["soh"],
62     "resistance": src["internal_resistance_ohm"],
63     "T": src["temperature_C"],
64     "C_rate": src["charge_rate_C"],
65     "dod": src["dod_pct"] / 100.0,
66     "condition": src["condition"],
67     "cond_id": src["cond_id"],
68     "stage": [get_stage(s) for s in src["soh"]],
69 })
70
71 # ---- 富特征保留列（规范未覆盖的建模字段，原样附带） ----
72 EXTRAS = ["battery_id", "soh_t", "capacity_Ah", "q_ref_Ah", "ccct_min", "cvct_min",
73     "ce", "phase_label", "knee_cycle", "rul_cycles", "is_eol", "reached_eol",
74     "chemistry", "cell_format", "rated_capacity_Ah", "nominal_voltage_V",
75     "discharge_rate_C", "soc_window", "charging_strategy", "v_upper_V",
76     "v_lower_V", "eol_threshold_pct", "batch_id", "is_abnormal",
77     "source_id", "source_ref"]
78 for c in EXTRAS:
79     spec[c] = src[c].values
80
81 # ---- 最终状态表：每电池最后一行（到达EOL）+ RUL/grade 占位 ----
82 last = src.sort_values(["cond_id", "cycle"]).groupby("cond_id").tail(1).copy()
83 final = pd.DataFrame({
84     "cycle": last["cycle"],
85     "SOH": last["soh"],
86     "resistance": last["internal_resistance_ohm"],
87     "T": last["temperature_C"],
88     "C_rate": last["charge_rate_C"],
89     "dod": last["dod_pct"] / 100.0,
90     "condition": last["condition"],
91     "cond_id": last["cond_id"],
92     "stage": [get_stage(s) for s in last["soh"]],
93     "RUL": np.nan, # 成员B计算后填入
94     "grade": "", # 成员C分级后填入
95 })
96 for c in EXTRAS:
97     final[c] = last[c].values
98
99 ts_path = os.path.join(DATA, "battery_timeseries.csv")
100 fs_path = os.path.join(DATA, "battery_final_states.csv")
101 spec.to_csv(ts_path, index=False, encoding="utf-8-sig")
102 final.to_csv(fs_path, index=False, encoding="utf-8-sig")
103
104 # ---- 摘要校验 ----
105 print(f"battery_timeseries : {len(spec):,} 行 x {spec.shape[1]} 列 -> {ts_path}")
106 print(f"battery_final_states: {len(final)} 行 x {final.shape[1]} 列 -> {fs_path}")
107 print(f"电池数(cond_id): {spec['cond_id'].nunique()} 工况串数: {spec['condition'].nunique()}")
108 print(f"dod 范围: {spec['dod'].min()} ~ {spec['dod'].max()} (小数)")
109 print(f"SOH 范围: {spec['SOH'].min():.4f} ~ {spec['SOH'].max():.4f}")
110 print("stage 分布:", spec["stage"].value_counts().to_dict())
111 print("condition 示例:", sorted(spec["condition"].unique())[:4], "...")
112 assert spec["dod"].between(0, 1).all(), "dod 必须为小数且在 [0,1]"
113 assert spec["SOH"].between(0.68, 1.01).all(), "SOH 越界"

```

```

114     print("校验通过 [OK]")
115
116
117 if __name__ == "__main__":
118     main()

```

程序 7: 数据格式规范转换脚本

附录 D 参考资料与知识库索引

4.1 参考文献

正文参考文献共 12 条（见前文 Bibliography，由 `paper/ref.bib` 管理，编号遵循 GB/T 7714），涵盖退化机理综述 [3, 6]、影响因子实验 [1, 2, 5, 7]、健康特征 [4]、寿命预测方法 [8, 9] 及本地精读文献 [10–12]。

4.2 知识库资料索引

支撑材料中支撑材料/参考资料/ 为备赛资料库与知识库，目录结构与关键内容如下（完整索引见支撑材料/参考资料/README_ 资料索引.md）：

目录 / 文件	内容
01_ 竞赛试题/	竞赛原题 + 题目解析与数据构建指南
02_ 竞赛规则/	论文格式规范、AI 工具使用规定、合规检查清单
03_A 题论文精读/	核心文献原文 + 详细精读（EEMD-GRU 寿命预测、退役电池快速聚类）
04_ 数学建模知识/	题型算法总览、AI 辅助建模提示词、论文写作规范、算法源代码库等知识库
05_ 工具速成/	Python 与 Matlab 数模速成、VS Code + LaTeX 统一配置指南
06_undermind 论文精读/	问题 1 相关 7 篇英文文献精读（退化机理、影响因子、健康特征）
数据汇总/	数据来源总览与逐条数据来源清单（含 PDF 位置）